RV College of Engineering

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

MAJOR PROJECT REPORT
(CS481P)

Submitted by,

Nikhil Vasu	**1RV22CS128**
Rohith Biradar	**1RV22CS164**
Suraj Chanaveeragoudra	**1RV22CS211**

Under the guidance of

Dr. Nagaraja G.S
Professor
Dept. of CSE
RV College of Engineering

Sanjana Ravindra Otihal
Assistant Professor
Dept. of CSE
RV College of Engineering

In partial fulfillment for the award of degree of

Bachelor of Engineering
in
Computer Science and Engineering

Department of CSE
2025-26

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

A Major Project Report (CS481P)

Submitted by,

Nikhil Vasu	**1RV22CS128**
Rohith Biradar	**1RV22CS164**
Suraj Chanaveeragoudra	**1RV22CS211**

Under the guidance of

Dr. Nagaraja G.S
Professor
Dept. of CSE
RV College of Engineering

Sanjana Ravindra Otihal
Assistant Professor
Dept. of CSE
RV College of Engineering

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Computer Science and Engineering

CERTIFICATE

Certified that the major project (CS481P) work titled ***Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection*** is carried out by **Nikhil Vasu** (**1RV22CS128**), **Rohith Biradar** (**1RV22CS164**) and **Suraj Chanaveeragoudra** (**1RV22CS211**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	**Head of the Department**	**Dean CSE Cluster**	**Principal**
Dr. Nagaraja G.S	Dr. Shanta Rangaswamy	Dr. Ramakanth Kumar P	Dr. K. N. Subramanya

External Viva

Name of Examiners	**Signature with Date**
1.	
2.	

DECLARATION

We, **Nikhil Vasu**, **Rohith Biradar** and **Suraj Chanaveeragoudra** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name **Signature**

1. Nikhil Vasu(1RV22CS128)
2. Rohith Biradar(1RV22CS164)
3. Suraj Chanaveeragoudra(1RV22CS211)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Nagaraja G.S**, Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our guide, **Sanjana Ravindra Otihal**, Assistant Professor, Department of , RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Dr. Chethana R Murthy**, Professor, Department of CSE and **Prof. Deepika Dash**, Assistant Professor, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

Our sincere thanks to **Dr. Ramakanth Kumar P**, Dean CSE Cluster, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Trade-Based Money Laundering (TBML) has emerged as a significant challenge within modern financial systems due to the increasing complexity and scale of global trade transactions. Conventional Anti-Money Laundering (AML) systems primarily rely on isolated rule-based monitoring frameworks that are often ineffective in identifying hidden transactional dependencies and multi-hop laundering typologies across interconnected banking networks. To address these limitations, this work developed a privacy-preserving Federated Heterogeneous Graph Neural Network (HGNN)-based framework for real-time multi-class TBML detection across distributed institutional environments.

The proposed framework modeled accounts, trade entities, SWIFT transactions, and trade documents as heterogeneous financial graphs using Memgraph. A hybrid analytical architecture integrating Graph Attention Network (GAT)-based convolutional layers and Long Short-Term Memory (LSTM)-based temporal sequence modeling was employed to capture relational topology and evolving transactional behavior. Federated learning using the Federated Averaging (FedAvg) aggregation strategy enabled collaborative cross-bank model optimization without centralized transactional data sharing. The implemented workflow incorporated Apache Kafka-based transaction streaming, graph construction, federated fraud inference, Suspicious Transaction Report (STR) generation, and regulator-facing compliance synchronization using Python, PyTorch Geometric, FastAPI, PostgreSQL, Memgraph, and React.js.

Experimental evaluation across distributed banking clients demonstrated stable federated convergence with Fraud-class Precision values approaching 0.98, F1-Scores nearing 0.84, and Precision–Recall Area Under Curve (PR-AUC) values exceeding 0.92. Concurrent scalability evaluation additionally maintained dashboard retrieval latency near 156–162 ms and fraud-filtering response latency near 148–159 ms under increasing institutional workloads, thereby validating the operational suitability of the proposed framework for scalable and privacy-preserving TBML detection. The framework can be extended into a fully integrated banking compliance platform with automated risk response, transaction control, and regulatory workflows. These enhancements would transform it from a monitoring system into a complete financial crime prevention ecosystem.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

AI Artificial Intelligence

AML Anti-Money Laundering

API Application Programming Interface

DBMS Database Management System

FedAvg Federated Averaging

FL Federated Learning

GAT Graph Attention Network

GCN Graph Convolutional Network

GNN Graph Neural Network

GUI Graphical User Interface

HGNN Heterogeneous Graph Neural Network

KYC Know Your Customer

ML Machine Learning

NLP Natural Language Processing

STR Suspicious Transaction Report

TBML Trade-Based Money Laundering

Chapter 1

Introduction to Trade-Based Money Laundering

CHAPTER 1

INTRODUCTION TO TRADE-BASED MONEY LAUNDERING

International trade and digital finance have made global transactions faster, but also harder to monitor. Criminals increasingly exploit this complexity to move illicit value through legitimate trade activity, exposing the limits of conventional Anti-Money Laundering (AML) systems. Trade-Based Money Laundering (TBML) is especially difficult to detect because the suspicious activity is hidden inside normal commercial exchanges. This creates a clear need for intelligent, scalable, and privacy-preserving methods that can uncover hidden transactional relationships in real time.

> *"For financial institutions, TBML is uniquely challenging because illicit funds are embedded within legitimate trade transactions, leaving traditional AML controls blind to the risks."*
>
> *– Dr. Sebastian Hetzler, IMTF Co-CEO [1]*

TBML works by blending illicit financial flows into ordinary trade transactions, which reduces the effectiveness of rule-based compliance checks and isolated transaction monitoring.

1.1 Overview of Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) refers to disguising the proceeds of crime through legitimate international trade transactions [2]. Instead of moving money directly, TBML hides illegal value inside import-export activities, invoice settlements, and cross-border trade records. Because these transactions involve multiple intermediaries, institutions, and jurisdictions, conventional AML systems struggle to identify suspicious patterns reliably.

1.1.1 Global Financial Crime Scenario

The growth of cross-border trade has made illicit activity harder to spot in normal financial flows. The United Nations Office on Drugs and Crime (UNODC) estimates that money laundering accounts for nearly 2% to 5% of global GDP, or about $800 billion to $2 trillion annually [3]. Europol also reports that authorities intercept less than 1% of

illicit financial flows [4]. These figures show how quickly laundering networks are evolving and why traditional monitoring tools are no longer enough.

As goods, services, and financial assets move across borders, criminal actors can hide suspicious transactions inside legitimate trade activity. Even as banks strengthen Know-Your-Customer (KYC) controls and transaction surveillance, TBML continues to exploit gaps in conventional monitoring.

Table 1.1: Key Statistics Related to Global Trade-Based Financial Crime

Financial Crime Metric	**Estimated Value**
Global GDP associated with money laundering annually	2% – 5%
Estimated annual TBML economic volume	$1.6T – $2.0T
Illicit flows linked to trade misinvoicing in developing economies	87%
Global interception rate of illicit financial flows	<1%
Reduction in false positive alerts using AI-driven monitoring systems	Up to 40%

1.1.2 Trade-Based Money Laundering

TBML allows criminal organizations to move illicit value through trade while avoiding direct cash movement. Common techniques include over-invoicing, under-invoicing, multiple invoicing, false goods descriptions, phantom shipments, and shell entities that distort trade records. Because these actions are spread across real trade operations, they are difficult for regulators and financial institutions to detect.

Global Financial Integrity reports that trade misinvoicing and related TBML activity account for nearly 87% of illicit financial flows from developing economies [5]. Industry estimates also suggest that about $1.6 trillion to $2 trillion in illicit value is hidden each year within trade invoices and commercial documents [1]. Together, these findings show how heavily organized crime depends on the international trade system.

1.1.3 Problem Area

Conventional AML systems mainly rely on predefined rules, threshold-based monitoring, and manual investigations. Although these approaches can identify isolated suspicious transactions, they often fail to capture the interconnected relationships involved in coordinated money laundering activities. TBML networks typically involve importers,

exporters, shell companies, offshore entities, logistics providers, and financial intermediaries operating across multiple jurisdictions, making suspicious patterns difficult to trace using traditional approaches. Existing machine learning methods also face limitations because they generally analyze transactions as independent events, often overlooking hidden structural and relational patterns within financial networks.

As a result, traditional systems frequently generate high false positives, increase investigation delays, and reduce overall detection efficiency. In addition, privacy and regulatory restrictions prevent institutions from freely sharing sensitive transactional data, making centralized fraud detection impractical. These challenges highlight the need for scalable and privacy-preserving analytical frameworks capable of modeling complex distributed financial ecosystems while effectively detecting sophisticated TBML activities.

1.2 Literature Review

The growth of global finance has led to a large body of AML and TBML research. Earlier work focused on rule-based monitoring, but newer studies increasingly use machine learning, graph analytics, and distributed learning to detect hidden patterns in financial data. This shift is important because TBML does not usually appear as a single suspicious transfer; instead, it is spread across invoices, counterparties, shipping records, account movements, and commercial relationships. For that reason, recent literature has moved from isolated transaction screening toward methods that can model interactions between entities and uncover structural patterns in the wider financial network.

Jiani Fan, Lwin Khin Shar, Ruichen Zhang, Ziyao Liu, Wenzhuo Yang, Dusit Niyato, and Kwok-Yan Lam [6] surveyed deep learning methods for AML and showed that CNNs, RNNs, and GNNs can improve anomaly detection. Their survey is useful because it presents how learning-based techniques can extract richer patterns than fixed rules alone, especially in environments with large transaction volumes. At the same time, the work remained centered on centralized data processing, which makes it less suitable for institutions that cannot share sensitive records directly. This limitation is especially relevant in TBML scenarios, where data from multiple organizations may be needed to understand the full laundering chain.

Rasmus Ingemann Tuffveson Jensen and Alexandros Iosifidis [7] evaluated standard machine learning models for financial anomaly detection, including Logistic Regression, Random Forests, and Gradient Boosting methods. Their results showed that traditional

supervised models can outperform static rule-based approaches when the data is well prepared and labeled. However, the study still treated transactions largely as independent events. In practice, laundering activity is often distributed across connected accounts, counterparties, and repeated trade interactions, so a purely transaction-level view can miss the broader relational structure.

Jie Song, Sijia Zhang, Junghoon Park, Yu Gu, and Ge Yu [8] proposed a dimension expansion framework for learning hidden money laundering activities within transaction networks. The method strengthened transaction representation by combining neighboring account information with contextual interaction features, which is a useful step toward graph-aware fraud detection. The study showed that incorporating relational information can improve the detection of concealed laundering behavior across financial networks. Even so, the approach still relied on manually engineered features, which can reduce flexibility when the structure of the network changes or when the system must operate across distributed institutions.

In another study, Jie Song, Sijia Zhang, Pengyi Zhang, Junghoon Park, Yu Gu, and Ge Yu [9] introduced a Social Attention Graph Neural Network framework for detecting illicit accounts in blockchain transaction systems. Their work demonstrated the value of attention mechanisms in highlighting the most informative neighbors during graph learning. It also reinforced the idea that graph neural networks can outperform traditional machine learning models in suspicious-account detection. However, the study was focused on blockchain transactions rather than trade documents or invoice-based laundering, so it does not fully address the heterogeneous relationships that appear in TBML cases.

Zhong Li, Xueting Yang, and Changjun Jiang [10] proposed a Multi-View Graph-Based Hierarchical Representation Learning framework for organized money laundering group detection. Their use of multiple graph views helped identify coordinated laundering communities and showed that hierarchical graph representations can capture more than simple pairwise links. The work is valuable because it moves beyond flat feature vectors and models fraud as a collaborative activity. Still, the approach depended on predefined graph relations and centralized repositories, which can limit use in privacy-sensitive environments where institutions must keep their own data local.

Abdul Haluk Batur Gezici and Emre Sefer [11] proposed AMLPD, an unsupervised Anti-Money Laundering detection framework based on Personalized PageRank and graph

diffusion. This work is important because it reduces dependence on labeled data, which is often expensive and incomplete in real banking systems. The method was able to capture hidden graph structure in sparse financial networks and demonstrated that unsupervised learning can be practical for anomaly detection. However, the lack of supervised optimization also led to more false positives, which can increase the workload of compliance teams and reduce trust in the system.

Md. Rezaul Karim, Felix Hermsen, Sisay Adugna Chala, Paola De Perthuis, and Avikarsha Mandal [12] investigated scalable semi-supervised graph learning architectures for large-scale financial transaction monitoring. Their evaluation of SkipGCN and EvolveGCN showed that graph learning can scale to bigger transaction datasets while still preserving useful classification performance. This makes the work relevant for real-world fraud monitoring systems that must handle large volumes of records. Even so, the framework assumed centralized graph access and did not directly solve the privacy and regulatory challenges that arise when multiple institutions need to collaborate.

Kyungmo Koo, Minyoung Park, and Byungun Yoon [13] introduced an autoencoder-based suspicious transaction detection framework using Risk-Based Approaches (RBA) for banking systems. Their work demonstrated how compact latent representations of transaction data can simplify fraud detection while supporting internal banking surveillance. The study presents a computationally efficient alternative to complex graph-based models. However, the approach mainly focused on detecting isolated account-level anomalies and did not address the interconnected laundering relationships commonly observed in large-scale trade ecosystems.

Overall, recent research shows a gradual transition from traditional rule-based AML systems toward graph-driven and machine learning-based approaches. Despite these advancements, several challenges still remain in areas such as heterogeneous graph modeling, real-time fraud analysis, privacy-preserving collaboration, and scalability in distributed financial environments. In addition, many existing approaches are designed for centralized banking or cryptocurrency datasets and do not fully capture the complex trade relationships associated with TBML. These limitations motivate the need for a unified framework that combines heterogeneous graph learning, federated intelligence, and real-time detection capabilities.

1.3 Motivation

The work, *"Federated Heterogeneous Graph Neural Networks for Real-Time Detection of Trade-Based Money Laundering"*, was selected to address the growing complexity of financial crime in modern trade ecosystems. Conventional AML systems often rely on rule-based checks and isolated transaction analysis, which makes them less effective at identifying coordinated laundering patterns spread across connected financial networks. In practice, this leads to more false positives, more manual review, and weaker detection of sophisticated TBML activity.

This project addresses those limitations by combining heterogeneous graph learning with federated intelligence to model transaction relationships while preserving institutional privacy. By using graph attention mechanisms and collaborative training across organizations, the framework supports fraud detection without sharing sensitive transactional data directly. It also adds automated Suspicious Transaction Report (STR) generation and dashboard-based monitoring to support real-time compliance analysis.

1.4 Problem Statement

Existing AML systems rely on rules and isolated transaction analysis, which limits their ability to detect TBML and hidden financial networks. Privacy and regulatory constraints also make cross-institution collaboration difficult. A scalable and privacy-preserving framework is therefore needed to model interconnected financial data and detect suspicious activity in real time.

1.5 Objectives

The objectives of the project are

1. To develop a Federated Heterogeneous Graph Neural Network framework for real-time detection of Trade-Based Money Laundering activities.
2. To model complex transactional relationships among financial entities using heterogeneous graph representation and graph attention mechanisms.
3. To generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions to support financial investigation and compliance processes.

4. To design an interactive dashboard for visualization of fraud predictions, transaction analysis, and real-time monitoring of suspicious activities.

1.6 Brief Methodology of the project

The proposed methodology builds a privacy-preserving graph-based framework for real-time TBML detection. Financial transaction data, trade records, customer information, and entity relationships are first collected and cleaned to create structured analysis datasets. These records are then converted into a heterogeneous graph where accounts, organizations, invoices, and transactions are linked as nodes and edges.

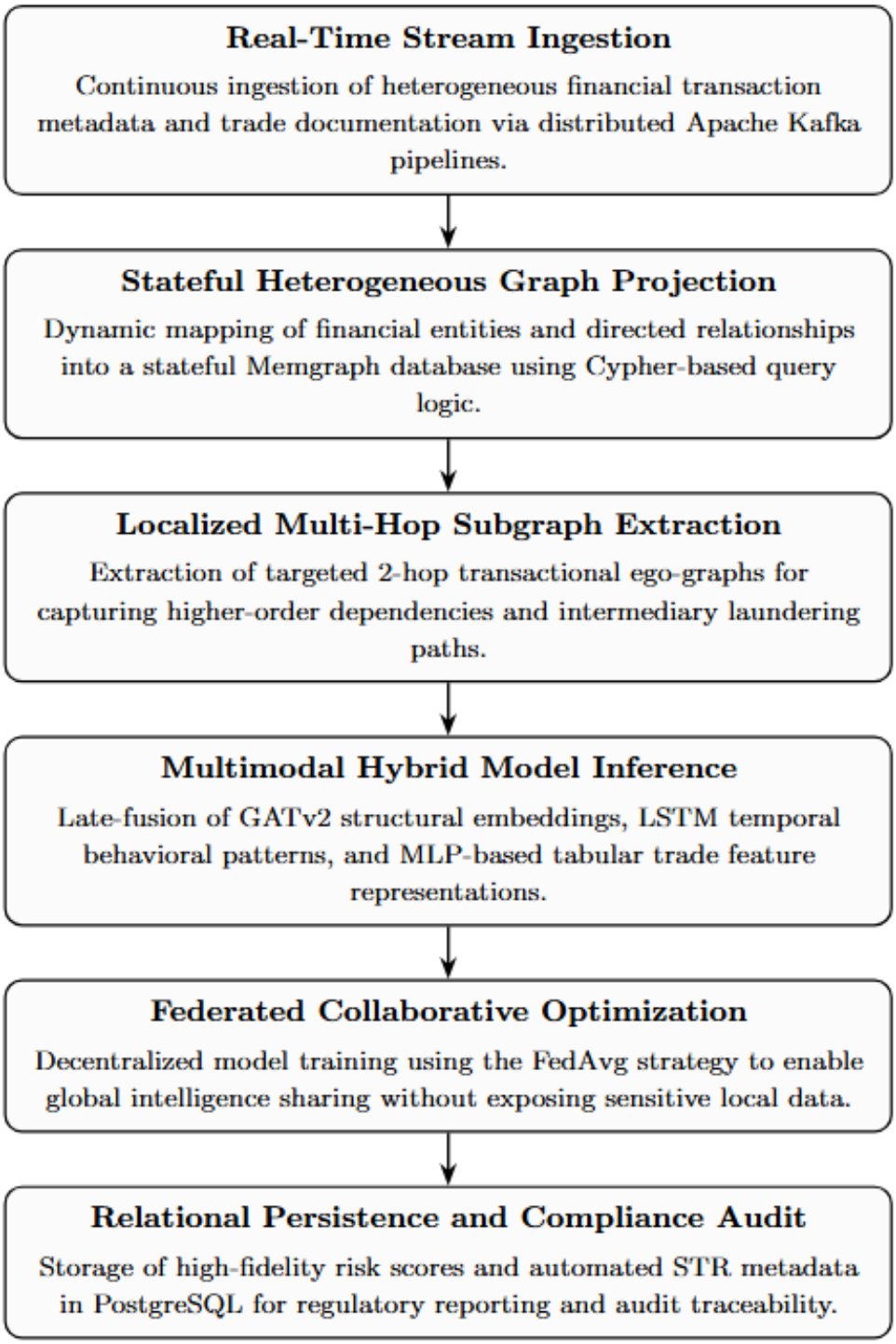

Figure 1.1: Proposed Methodology for TBML Detection

Graph attention mechanisms are used to capture hidden relationships and suspicious behavioral patterns within the financial ecosystem. Federated learning is then added so that institutions can train collaboratively without exposing sensitive local data. The detected suspicious activities are finally used to generate STRs and support real-time

monitoring through an interactive dashboard.

1.7 Assumptions and Constraints of the Work

Assumptions

- The financial transaction records, trade invoices, and customer-related information used for analysis are assumed to be sufficiently structured, consistent, and preprocessed for effective model training and evaluation.
- It is assumed that participating financial institutions maintain similar transactional formats and standardized trade-related attributes required for graph construction and suspicious activity analysis.
- The generated financial datasets are assumed to contain sufficient normal and suspicious transactional patterns necessary for learning hidden behavioral relationships associated with Trade-Based Money Laundering activities.
- It is assumed that heterogeneous graph representations can effectively model relationships among accounts, organizations, transactions, invoices, and trade entities for identifying suspicious financial behavior.

Constraints

- Limited access to real-world Trade-Based Money Laundering datasets due to institutional privacy regulations and confidentiality policies may affect the diversity of transactional data used for evaluation.
- Large-scale graph construction and real-time transaction analysis may increase computational complexity and memory requirements during fraud detection operations.
- The performance of the proposed framework depends significantly on the quality, completeness, and accuracy of the financial data available for model training and testing.
- The proposed framework is validated primarily under controlled academic and experimental conditions and may require additional optimization for large-scale industrial deployment.

1.8 Organization of the Report

This report is organized as follows.

- Chapter 2 discusses the theoretical foundations associated with Trade-Based Money Laundering detection, including Anti-Money Laundering systems, heterogeneous graph modeling, Graph Neural Networks, and federated learning methodologies relevant to the proposed framework.
- Chapter 3 illustrates the Software Requirement Specification (SRS) of the proposed system, including functional requirements, system constraints, software and hardware requirements, and operational specifications.
- Chapter 4 presents the overall system architecture and high-level design of the proposed framework, including workflow representations, component interactions, data flow mechanisms, and dashboard architecture.
- Chapter 5 elaborates the detailed system design methodology, including graph construction mechanisms, fraud-detection workflows, STR generation pipelines, and compliance-monitoring integration.
- Chapter 6 describes the implementation procedures adopted for the proposed framework, including dataset preprocessing, graph generation, backend development, federated model integration, and dashboard deployment.
- Chapter 7 evaluates the testing and validation procedures performed on the proposed system, including functional validation, fraud prediction analysis, dashboard verification, and model performance evaluation.
- Chapter 8 analyzes the experimental results and performance outcomes of the proposed framework using fraud-classification metrics, scalability observations, and system-level analytical evaluation.
- Chapter 9 summarizes the overall conclusions of the proposed work while additionally discussing the current limitations and potential future enhancements of the framework.

Chapter 2

Theory and Fundamentals of Project

CHAPTER 2

THEORY AND FUNDAMENTALS OF PROJECT

Developing an intelligent framework for Trade-Based Money Laundering detection requires a strong foundation in financial crime analysis, graph theory, and machine learning techniques. This chapter presents the fundamental concepts associated with Trade-Based Money Laundering, Graph Neural Networks, Graph Attention mechanisms, heterogeneous graph representation, and federated learning. The chapter further discusses the computational principles involved in relational transaction modeling, privacy-preserving learning, and real-time fraud analytics. These concepts collectively form the theoretical foundation for the proposed fraud detection framework.

2.1 Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) is a sophisticated form of financial crime in which illicit funds are transferred and concealed through seemingly legitimate international trade activities. Unlike conventional laundering methods that primarily depend on direct financial transactions, TBML exploits import-export systems, trade invoices, shipping documents, and cross-border commercial operations to disguise the movement of illegal funds. Due to the increasing implementation of strict Anti-Money Laundering (AML) regulations and Know Your Customer (KYC) policies, criminal organizations have increasingly shifted toward trade-oriented laundering mechanisms that are comparatively difficult to detect using traditional monitoring systems.

The complexity of modern global trade ecosystems, combined with the involvement of multiple financial entities and transactional relationships, has made TBML detection a major challenge for financial institutions and regulatory agencies. Consequently, intelligent analytical techniques capable of identifying hidden transactional dependencies and suspicious financial behavior have become increasingly important in modern Anti-Money Laundering systems.

2.1.1 Introduction to TBML

Trade-Based Money Laundering primarily involves the manipulation of international trade transactions to illegally transfer value between entities located across different countries. Criminal organizations often misuse trade documentation, invoice values, shipment quantities, and commercial agreements to move illicit funds while making the transac-

tions appear legitimate within global financial systems. Since these laundering activities are embedded within regular import-export operations, identifying suspicious behavior becomes highly challenging for financial institutions and regulatory agencies.

In many TBML operations, colluding buyers and sellers intentionally manipulate the declared value of traded goods to transfer illegal financial assets across borders. Techniques such as over invoicing, under invoicing, phantom shipments, and multiple invoicing are commonly used to conceal the movement of illicit proceeds. Such laundering mechanisms exploit the complexity and high transactional volume of international trade systems, making traditional rule-based Anti-Money Laundering approaches comparatively ineffective in detecting hidden financial relationships.

Figure 2.1 illustrates a typical Trade-Based Money Laundering workflow involving collusion between exporters and importers across different jurisdictions [14].

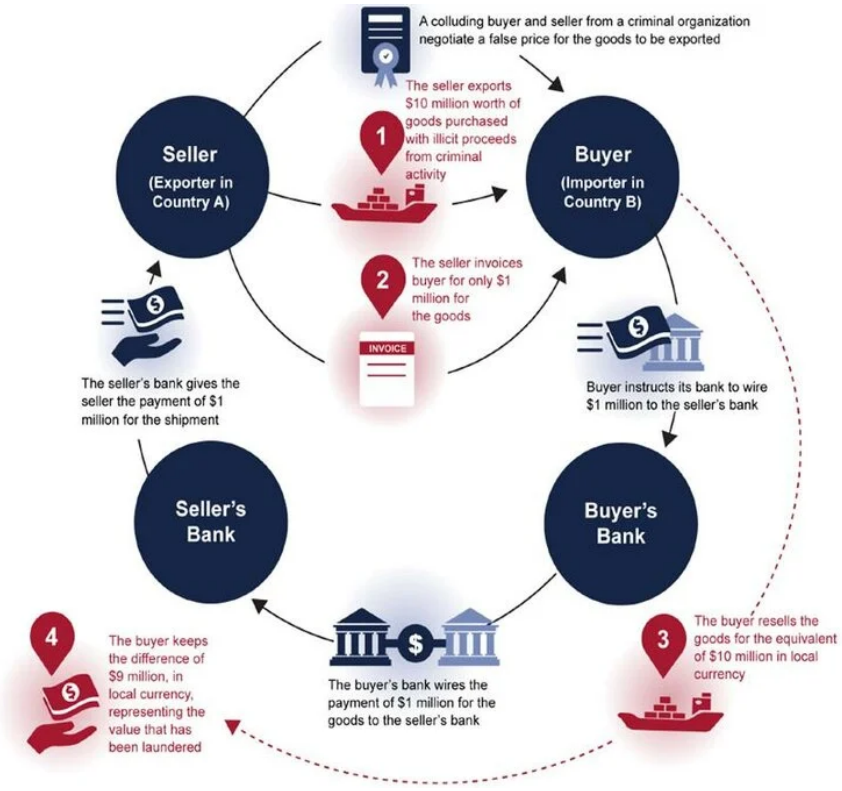

Figure 2.1: Trade-Based Money Laundering through trade misinvoicing and open-account transactions

2.1.2 Common Techniques Used in TBML

Trade-Based Money Laundering (TBML) involves manipulating trade documents, shipment details, and financial transactions to move illicit funds across borders while

making them appear legitimate. Criminal organizations exploit the complexity of international trade systems to avoid detection and conceal illegal financial activities.

Over Invoicing

Over invoicing occurs when the declared value of goods is intentionally set higher than the actual market price. This allows excess money to be transferred under the appearance of legitimate trade payments.

Under Invoicing

Under invoicing involves declaring goods at a lower value than their actual market price. This technique is commonly used to hide profits, evade taxes, and move funds secretly across borders.

Multiple Invoicing

Multiple invoicing refers to using the same trade invoice for several financial transactions related to a single shipment. This helps criminals transfer illicit funds through different banking channels.

Phantom Shipments

Phantom shipments occur when payments are made for goods that are never actually shipped or delivered. Fake trade and shipping documents are used to create the appearance of legitimate trade activity.

Misrepresentation of Goods

Misrepresentation of goods involves providing false information about the type, quantity, or quality of traded products in invoices and trade documents to manipulate trade values and conceal suspicious activities.

2.1.3 Challenges in Detecting TBML

Detecting TBML is difficult because modern trade systems are highly complex and interconnected. Unlike traditional financial fraud, TBML activities are often hidden within legitimate business transactions, making suspicious behavior hard to identify. Criminal networks use shell companies, intermediaries, multiple jurisdictions, and layered transactions to move illicit funds while appearing as normal trade operations.

Traditional AML systems mainly rely on rule-based monitoring and isolated transaction analysis. Although these methods can detect predefined suspicious patterns, they of-

ten fail to identify hidden relationships between entities involved in large trade networks. In addition, the growing volume of cross-border transactions and evolving laundering techniques make detection even more challenging. Privacy and regulatory restrictions also limit the sharing of sensitive financial data between institutions, creating the need for scalable and privacy-preserving TBML detection frameworks.

2.2 Algorithms Used

This section presents the major algorithms and deep learning techniques used in the proposed system for traffic prediction and autonomous vehicle navigation. The project combines graph-based learning and sequential modeling approaches to effectively analyze complex traffic patterns and temporal dependencies. Graph Attention Network v2 (GATv2) is utilized to capture spatial relationships between connected road networks through dynamic attention mechanisms, while Long Short-Term Memory (LSTM) is employed to learn temporal traffic variations and long-term sequential dependencies. Together, these algorithms improve traffic prediction accuracy, congestion analysis, and intelligent decision-making within the proposed framework.

2.2.1 Graph Attention Network v2 (GATv2)

Graph Attention Network v2 (GATv2) is a Graph Neural Network (GNN) architecture that uses a dynamic attention mechanism to learn relationships between connected nodes in graph data. It improves feature aggregation by automatically identifying the importance of neighboring nodes. GATv2 is widely used in applications such as traffic prediction, recommendation systems, and autonomous vehicle navigation.

Introduction

Graph Attention Network v2 (GATv2) is an advanced Graph Neural Network (GNN) architecture designed for learning on graph-structured data using a dynamic attention mechanism. It improves upon the original GAT model by overcoming the limitation of static attention and enabling better learning of relationships between connected nodes. GATv2 automatically identifies the importance of neighboring nodes during feature aggregation, making it highly effective for applications such as traffic prediction, recommendation systems, social network analysis, fraud detection, and autonomous vehicle navigation.

Fundamental Concept of GATv2

The primary objective of GATv2 is to generate node embeddings by aggregating information from neighboring nodes using a dynamic attention mechanism. For each node, the model learns which neighboring nodes are important and assigns suitable attention weights before combining their features.

Architecture and Working of GATv2

The overall architecture and workflow of GATv2 are shown in Figure 1.1.

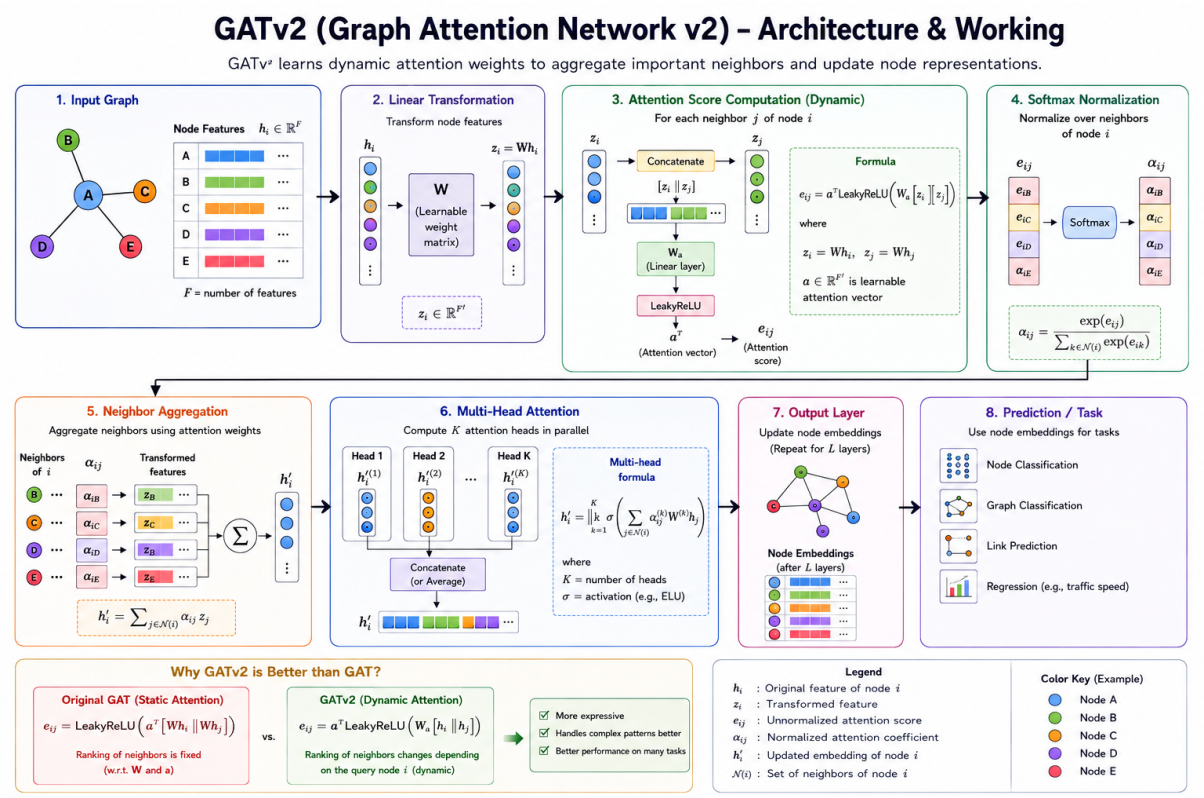

Figure 2.2: GATv2 Architecture and Working

The architecture of GATv2 consists of graph input processing, feature transformation, attention computation, normalization, neighbor aggregation, multi-head attention, and prediction generation.

Step 1: Input Graph: The process begins with a graph containing nodes, edges, and node feature vectors. Each node feature vector is represented as:

$$h_i \in \mathbb{R}^F$$

where h_i denotes the feature vector of node i and F represents the number of input features. In traffic prediction systems, nodes may represent road intersections while edges represent road connectivity.

Step 2: Linear Transformation: The input node features are transformed using a learnable weight matrix:

$$z_i = Wh_i \tag{2.1}$$

where W is the learnable weight matrix and z_i is the transformed feature representation. This transformation improves feature learning before attention computation.

Step 3: Attention Score Computation: For every connected node pair (i, j), the attention score is computed as:

$$e_{ij} = a^T \text{LeakyReLU}\left(W[h_i \| h_j]\right) \tag{2.2}$$

where a is the learnable attention vector and e_{ij} represents the attention score between neighboring nodes.

Step 4: Softmax Normalization: The attention scores are normalized using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \tag{2.3}$$

where α_{ij} denotes the normalized attention coefficient and $\mathcal{N}(i)$ represents the neighboring nodes of node i.

Step 5: Neighbor Aggregation: The neighboring node features are aggregated using weighted summation:

$$h_i' = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j \tag{2.4}$$

where h_i' is the updated node representation.

Step 6: Multi-Head Attention: GATv2 employs multiple attention heads operating in parallel:

$$h_i' = \Big\Vert_{k=1}^{K} \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} h_j\right) \tag{2.5}$$

where K represents the number of attention heads and $\|$ denotes concatenation. Multi-head attention improves representation learning and robustness.

Step 7: Output Layer and Prediction: The final node embeddings are passed to the output layer for tasks such as node classification, graph classification, link prediction, and regression.

Advantages of GATv2

GATv2 provides a dynamic attention mechanism that improves feature aggregation and representation learning compared to the original GAT architecture. It automatically identifies important neighboring nodes, performs effectively on irregular graph data, and improves robustness through multi-head attention, making it highly suitable for traffic prediction and autonomous navigation applications.

Limitations of GATv2

Despite its advantages, GATv2 requires high computational and memory resources due to attention computations on large graphs. Training time increases with graph size, and deep GATv2 architectures may suffer from over-smoothing and scalability issues for extremely large datasets.

GATv2 vs GAT

Graph Attention Network v2 (GATv2) was introduced as an improvement over the original Graph Attention Network (GAT) to overcome the limitation of static attention mechanisms. While both architectures use attention-based aggregation for graph representation learning, GATv2 provides a more expressive and adaptive attention mechanism.

In the original GAT architecture, the attention mechanism is defined as:

$$e_{ij} = \text{LeakyReLU}\left(a^T [Wh_i \| Wh_j]\right) \tag{2.6}$$

In GATv2, the order of operations is modified as:

$$e_{ij} = a^T \text{LeakyReLU}\left(W[h_i \| h_j]\right) \tag{2.7}$$

This modification allows the attention scores to dynamically depend on both source and neighboring nodes, improving feature interaction learning and representation capability.

Table 2.1: Comparison Between GAT and GATv2

Feature	**GAT**	**GATv2**
Attention Type	Static Attention	Dynamic Attention
Neighbor Ranking	Fixed	Context-dependent
Expressiveness	Limited	More expressive
Flexibility	Lower	Higher
Performance	Good	Improved on many graph tasks

GATv2 is preferred over GAT because its dynamic attention mechanism enables better learning of complex graph relationships, resulting in improved feature aggregation, representation learning, and prediction accuracy in graph-based applications.

2.2.2 Long Short-Term Memory (LSTM)

Introduction to LSTM

Long Short-Term Memory (LSTM) is an advanced Recurrent Neural Network (RNN) architecture designed to learn long-term dependencies in sequential data. It was introduced to overcome the vanishing gradient problem faced by traditional RNNs through the use of memory cells and gating mechanisms. LSTM can selectively remember, update, and forget information, making it highly effective for applications such as traffic prediction, speech recognition, natural language processing, and time-series forecasting.

Fundamental Concept of LSTM

The primary objective of LSTM is to preserve important sequential information over long time intervals using memory cells and gating mechanisms. The architecture controls information flow through input, forget, and output gates, enabling efficient learning of temporal dependencies.

Architecture and Working of LSTM

The overall workflow of LSTM consists of gate computation, memory cell update, and hidden state generation.

Input Gate, Forget Gate, and Output Gate

At each time step, the current input and previous hidden state are used to compute the input, forget, and output gates using sigmoid activation functions. These gates regulate information flow into and out of the memory cell.

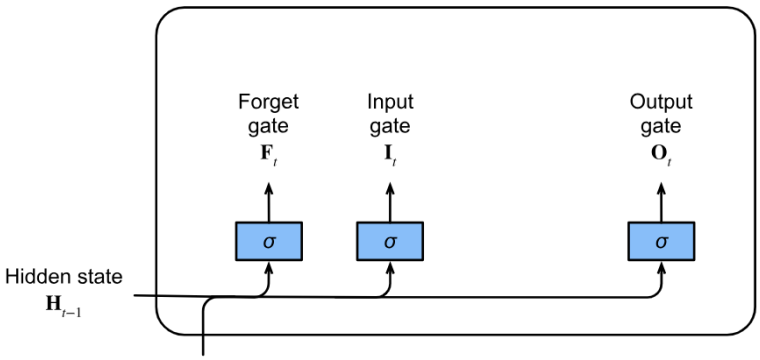

Figure 2.3: Computing the input gate, forget gate, and output gate in an LSTM model

The gates are mathematically represented as:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t\mathbf{W}_{xi} + \mathbf{H}_{t-1}\mathbf{W}_{hi} + \mathbf{b}_i)\\ \mathbf{F}_t &= \sigma(\mathbf{X}_t\mathbf{W}_{xf} + \mathbf{H}_{t-1}\mathbf{W}_{hf} + \mathbf{b}_f)\\ \mathbf{O}_t &= \sigma(\mathbf{X}_t\mathbf{W}_{xo} + \mathbf{H}_{t-1}\mathbf{W}_{ho} + \mathbf{b}_o)\end{aligned} \tag{2.8}$$

where $\mathbf{I}_t$, $\mathbf{F}_t$, and $\mathbf{O}_t$ represent the input, forget, and output gates respectively, while $\mathbf{X}_t$ and $\mathbf{H}_{t-1}$ denote the current input and previous hidden state.

Input Node

The candidate memory information is computed using the `tanh` activation function as:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t\mathbf{W}_{xc} + \mathbf{H}_{t-1}\mathbf{W}_{hc} + \mathbf{b}_c) \tag{2.9}$$

where $\tilde{\mathbf{C}}_t$ represents the candidate memory state.

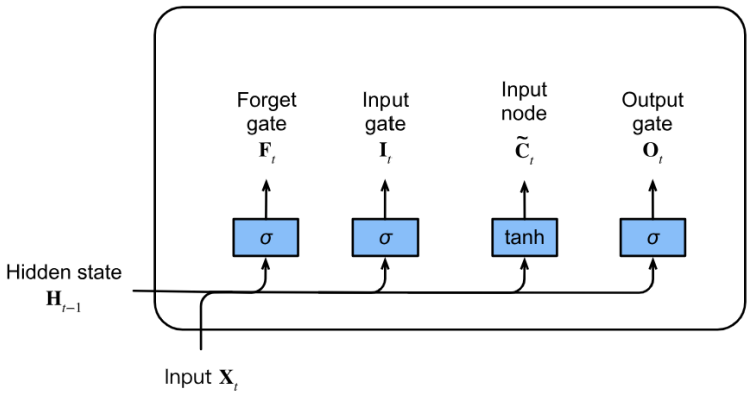

Figure 2.4: Computing the input node in an LSTM model

Memory Cell Internal State

The memory cell state is updated using the forget gate and input gate as:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \tag{2.10}$$

where $\mathbf{C}_t$ represents the updated memory cell state and $\odot$ denotes element-wise multiplication. This mechanism helps preserve important long-term information while discarding irrelevant data.

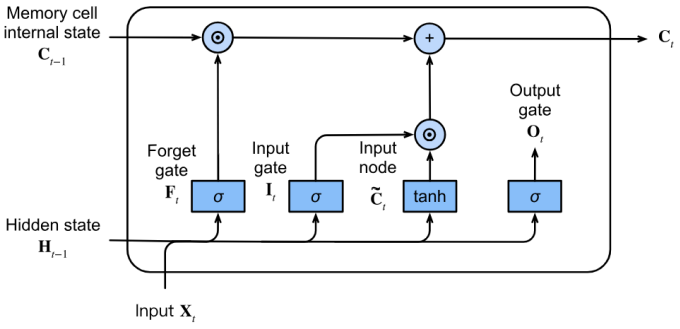

Figure 2.5: Computing the memory cell internal state in an LSTM model

Hidden State

The hidden state output is computed as:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \tag{2.11}$$

where $\mathbf{H}_t$ represents the hidden state at time step t. The output gate controls how much information from the memory cell is passed to the next layer.

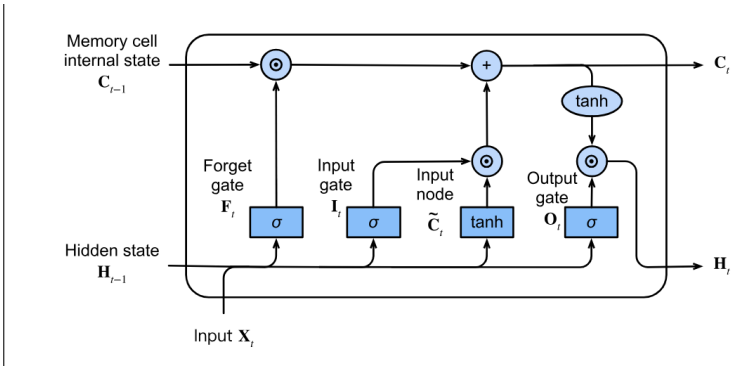

Figure 2.6: Computing the hidden state in an LSTM model

Advantages of LSTM

LSTM effectively solves the vanishing gradient problem and learns long-term sequential dependencies using memory cells and gating mechanisms. It performs efficiently on sequential and time-series data and is widely used in applications such as traffic prediction, speech recognition, machine translation, and forecasting.

Limitations of LSTM

Despite its advantages, LSTM is computationally expensive due to multiple gates and memory operations. Training requires higher computational resources and longer execution time, and performance may decrease for extremely long sequences or insufficient training data.

Source: Adapted from "Dive into Deep Learning, Long Short-Term Memory (LSTM)."

2.3 Federated Learning

Federated Learning (FL) is a decentralized machine learning approach where multiple devices or organizations collaboratively train a shared model without exchanging raw data. Each client performs local training on its own dataset and shares only model parameters with a central server, enabling privacy-preserving collaborative learning across distributed environments.

2.3.1 Introduction

Traditional machine learning systems rely on centralized training, where data from multiple users or organizations is collected and stored on a central server for model development. Although effective, this approach raises significant concerns related to data privacy, security, and regulatory compliance. Federated Learning addresses these chal-

lenges by enabling collaborative model training without sharing confidential or sensitive data.

In Trade-Based Money Laundering (TBML) detection systems, organizations such as banks and financial institutions can locally train models using transaction records and suspicious trade patterns while sharing only model parameters with a central server. Introduced by Google in 2016 for Gboard applications, Federated Learning has emerged as an important privacy-preserving approach for distributed machine learning systems .

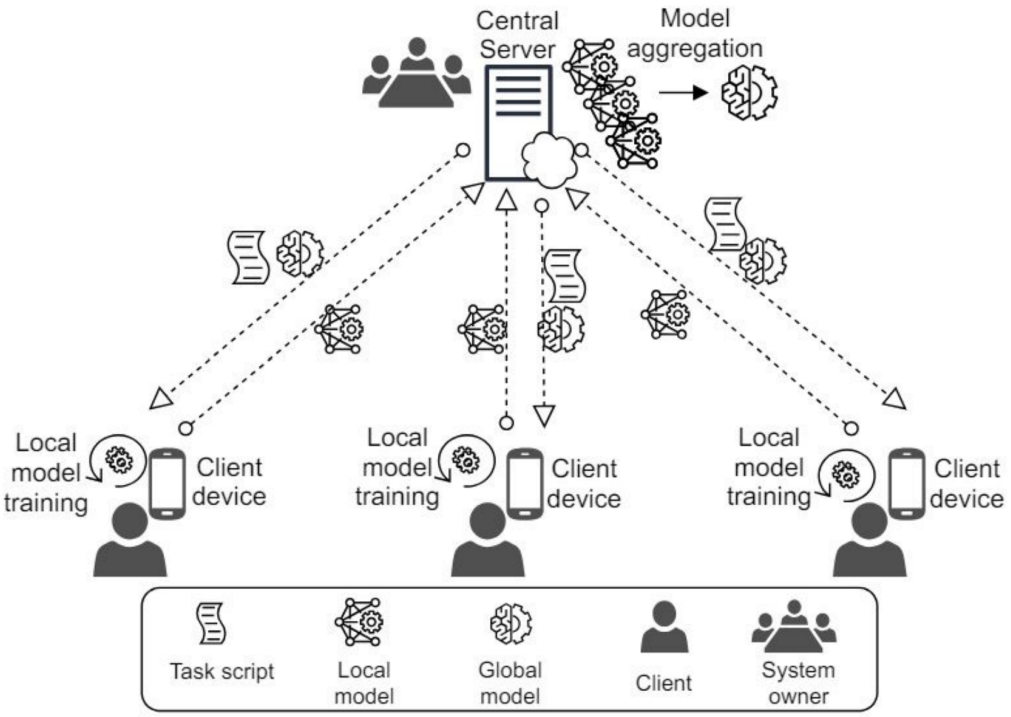

Figure 2.7: General Outlook of Federated Learning

Federated Learning also reduces communication overhead because only compact model updates are exchanged instead of complete datasets. This makes FL suitable for large-scale distributed environments involving mobile devices, IoT systems, edge devices, and financial institutions [15].

2.3.2 Working of Federated Learning

Federated Learning operates through iterative client-server interactions known as federated rounds. During each round, the central server distributes the current global model to selected clients, local training is performed independently on local datasets, and the updated model parameters are aggregated to generate an improved global model.

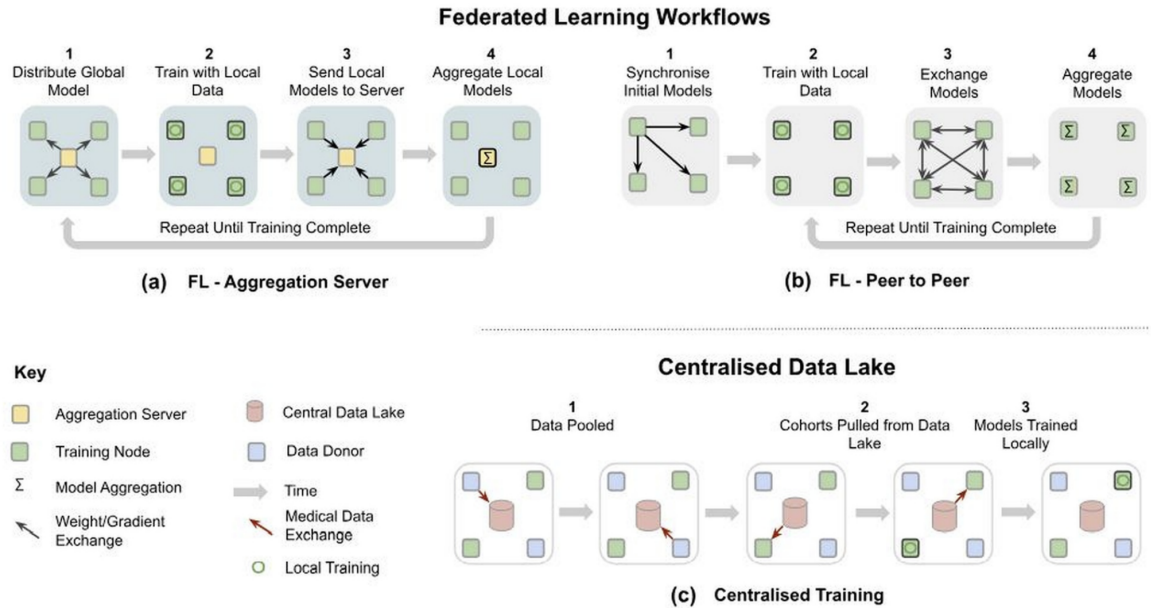

Figure 2.8: Federated Learning Workflows

Step 1: Initialization An initial machine learning model is selected and initialized at the central server. Client devices or institutional servers are then prepared to participate in local training.

Step 2: Client Selection During each federated round, only a subset of clients is selected for training. These clients receive the latest global model parameters from the server.

Step 3: Local Training Each selected client trains the received model locally using its private dataset without sharing raw transactional or customer information.

Step 4: Reporting and Aggregation After local training, clients send only model parameters or gradients to the aggregation server. The server combines these updates using aggregation algorithms such as Federated Averaging (FedAvg).

The Federated Averaging process is represented as:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{(t+1)} \tag{2.12}$$

where w_{t+1} represents the updated global model, $w_k^{(t+1)}$ denotes the local model of client k, n_k represents the number of training samples at client k, and n denotes the total

number of training samples across all clients.

Step 5: Termination The federated training process continues iteratively until the desired accuracy, convergence criteria, or maximum communication rounds are achieved.

2.3.3 Advantages of Federated Learning

Federated Learning improves data privacy and security by ensuring that raw data remains within local devices or organizations. It supports regulatory compliance, enables collaborative intelligence without direct data sharing, reduces communication overhead, and improves model generalization using diverse distributed datasets. FL is also scalable and supports real-time learning on edge devices and organizational servers.

2.3.4 Limitations of Federated Learning

Despite its advantages, Federated Learning introduces challenges such as communication delays, high computational requirements, non-i.i.d. data distributions, client failures, synchronization issues, and security threats such as model poisoning and backdoor attacks. Variations in client hardware capabilities and unstable network connections can also affect training efficiency and global model convergence.

2.4 User Interface

Modern financial monitoring platforms increasingly rely on scalable web application technologies to support real-time transaction analysis, fraud monitoring, and interactive visualization of financial activities. The rapid growth of digital banking and online financial services has significantly increased the demand for responsive, secure, and data-driven monitoring systems capable of handling large volumes of transactional information efficiently. Consequently, modern Anti-Money Laundering systems employ component-based web architectures and client-server communication models to provide efficient user interaction and real-time analytical capabilities.

Advanced frontend frameworks and backend service architectures play a critical role in enabling modular application development, dynamic rendering, secure authentication, and seamless integration with fraud detection engines. Such technologies support the visualization of suspicious transactional activities, alert generation, role-based access control, and automated compliance workflows within financial institutions. The integration of scalable user interface technologies with intelligent analytical systems further enhances

operational efficiency and supports real-time financial crime investigation processes.

2.4.1 Modern Web Application Frameworks

Modern web application frameworks provide the architectural foundation required for developing scalable, responsive, and interactive software systems. Traditional web applications primarily relied on static page rendering and tightly coupled server-side architectures, which limited dynamic user interaction and real-time data processing capabilities. In contrast, modern frameworks adopt component-based and asynchronous rendering approaches that improve application scalability, maintainability, and user experience within large-scale distributed systems.

Frontend frameworks such as React.js enable the development of reusable and modular user interface components capable of dynamically updating application states without requiring complete page reloads. Such frameworks employ virtual Document Object Model (DOM) mechanisms and state-driven rendering principles to efficiently manage user interactions and real-time data visualization. This architectural approach significantly improves responsiveness and enables scalable dashboard development for data-intensive applications such as financial monitoring systems.

Server-side rendering and hybrid rendering frameworks such as Next.js further enhance modern web application performance by combining client-side interactivity with optimized server-side content delivery. These frameworks support efficient routing, dynamic data fetching, and improved application scalability while reducing rendering overhead for large-scale systems. Consequently, modern web application frameworks have become an essential foundation for building secure, scalable, and real-time financial monitoring platforms.

2.4.2 Component Lifecycle and Reactivity

Modern frontend frameworks such as React and Next.js utilize reactive rendering mechanisms to synchronize user interfaces with continuously changing application states and backend responses. Within the proposed TBML detection framework, this reactive architecture enables efficient visualization of transaction analytics, fraud alerts, compliance activities, and real-time monitoring information without requiring complete interface reloading. The component-based design additionally improves scalability, maintainability, and responsiveness within large-scale financial monitoring environments.

The lifecycle of application components generally involves rendering, state initialization, reactive updates, synchronization, and resource cleanup operations during execution. The major lifecycle stages associated with the proposed dashboard architecture are summarized as follows:

Server-Side Rendering: Initially, dashboard components are rendered within the server environment where transaction summaries, analytical data, and monitoring information are processed before delivery to the client browser. This approach improves initial loading performance and reduces unnecessary client-side computational overhead.

Hydration and Client Initialization: After the interface is loaded within the browser, React hydration enables interactive client-side functionalities including event handling, asynchronous API communication, local state initialization, and real-time dashboard services.

Reactive State Updates: As transaction states, fraud alerts, and analytical outputs change, React selectively updates only the affected interface components using Virtual DOM reconciliation mechanisms instead of re-rendering the complete dashboard.

Dynamic Synchronization: The framework continuously synchronizes updated transaction information, compliance alerts, and monitoring analytics with the dashboard interface to support real-time fraud visualization and analytical consistency.

Resource Cleanup and Unmounting: When dashboard components are removed from the active interface, temporary states, subscriptions, and allocated resources are released automatically to maintain application stability and efficient memory utilization.

Such lifecycle management principles are important within modern financial monitoring systems where continuous transactional updates, fraud notifications, and analytical visualizations must be processed efficiently while maintaining scalable and responsive user interaction.

2.4.3 Data Visualization

Understanding the theoretical aspects of data visualization is important for analyzing large volumes of financial transactions and interpreting suspicious transactional behavior within real-time monitoring systems. Visualization of analytical outputs enables financial institutions and compliance analysts to identify fraud patterns, transaction anomalies,

risk distributions, and suspicious entity relationships more efficiently when compared with conventional tabular analysis methods. Interactive monitoring dashboards further improve situational awareness by enabling continuous observation of fraud alerts, transaction flows, and compliance activities without interrupting real-time analytical processes. This work utilizes a web-based dashboard architecture for financial monitoring, fraud visualization, and compliance-oriented analytical operations using modern frontend technologies and visualization frameworks.

Frontend Framework: Next.js is used for developing responsive and modular dashboard interfaces capable of supporting scalable financial monitoring applications. The framework provides server-side rendering, dynamic routing, and smooth integration with backend analytical services operating within the proposed TBML detection environment.

Analytical Visualization Components: Recharts is utilized to visualize transaction analytics, fraud classifications, risk-score distributions, and temporal financial trends through interactive graphs and dashboard components. These visualizations assist investigators and analysts in interpreting suspicious transactional behavior more efficiently during monitoring operations.

Role-Based Dashboard Interfaces: The dashboard architecture incorporates role-based access mechanisms to provide controlled access to fraud investigation workflows, Suspicious Transaction Report generation, real-time monitoring services, and institutional compliance operations according to user responsibilities and operational privileges.

Interactive Monitoring Services: Interactive dashboard panels support continuous monitoring of suspicious transactions, fraud alerts, transaction histories, and prediction results generated by the analytical framework. The monitoring environment additionally supports filtering, notification handling, export functionalities, and compliance-oriented investigation workflows for institutional analysts and regulatory authorities.

The detailed implementation and integration of these visualization technologies within the proposed TBML detection framework are discussed further in the system implementation chapter.

2.5 Summary

In this chapter, the theoretical foundations associated with Trade-Based Money Laundering (TBML) detection were presented, including laundering methodologies, limitations

of conventional Anti-Money Laundering systems, graph-based financial modeling, heterogeneous Graph Neural Networks, Graph Attention mechanisms, and federated learning concepts. The chapter additionally discussed real-time financial monitoring architectures and visualization systems used for fraud analytics and compliance operations. These concepts collectively establish the analytical and computational foundation for the proposed privacy-preserving TBML detection framework presented in the subsequent chapters.

Chapter 3

Design of the Proposed TBML Detection Framework

CHAPTER 3

DESIGN OF THE PROPOSED TBML DETECTION FRAMEWORK

This chapter describes the design of the proposed TBML detection framework, unifying software requirements, high-level architecture, and module-level design. It covers operational requirements and user roles, the system architecture and workflows, graph-construction techniques, the federated learning environment, data flow representations, and the analytical modules that enable detection, investigation, and reporting.

3.1 Software Requirements Specifications

The software requirements specifications of the proposed TBML detection framework define the operational functionalities, performance expectations, software dependencies, and computational resources required for efficient execution of the system. These specifications ensure that the framework can support real-time fraud analysis, scalable transaction monitoring, secure user interaction, and intelligent compliance management within financial institutions. The detailed software requirements are categorized into functional requirements, non-functional requirements, software dependencies, and hardware requirements as described in the following subsections.

3.1.1 Overall Description

The proposed framework is designed as a privacy-preserving financial monitoring platform for identifying suspicious Trade-Based Money Laundering activities across large-scale transactional ecosystems. The system integrates heterogeneous graph-learning mechanisms, federated analytical workflows, role-based monitoring dashboards, and automated Suspicious Transaction Report (STR) generation within a centralized compliance-monitoring environment. The framework supports real-time transaction monitoring, graph-based fraud classification, institutional investigation workflows, and regulator-side analytical visualization for detecting hidden laundering relationships across interconnected financial networks.

Product Perspective

The proposed TBML detection framework functions as an intelligent financial monitoring and fraud analysis platform capable of integrating machine learning-based anomaly

detection with interactive compliance monitoring systems. The architecture combines heterogeneous graph representation, Graph Neural Network-based analytical models, federated learning mechanisms, and modern web-based visualization technologies within a unified system environment.

The system represents financial entities such as accounts, organizations, invoices, and transactions as interconnected graph structures for relational analysis. Fraud detection modules analyze these relationships to identify suspicious transactional dependencies and hidden laundering patterns that are difficult to detect using conventional rule-based Anti-Money Laundering systems. The framework additionally integrates role-based dashboards for transaction monitoring, fraud visualization, alert management, and automated compliance reporting within financial institutions.

Product Functions

The primary functionalities supported by the proposed framework are summarized as follows:

- **Transaction Monitoring:** The system continuously monitors financial transactions and trade-related activities to identify suspicious transactional behavior within large-scale financial ecosystems.
- **Graph-Based Financial Analysis:** Financial entities such as accounts, organizations, invoices, and transactions are represented as interconnected graph structures for relational analysis and hidden dependency detection.
- **Fraud Classification and Risk Prediction:** Graph Neural Network-based analytical models classify suspicious transactions and generate fraud risk predictions based on transactional relationships and behavioral patterns.
- **Federated Learning Support:** The framework supports collaborative model training across multiple institutional environments while preserving sensitive financial data privacy.
- **Real-Time Dashboard Visualization:** Interactive monitoring dashboards provide visualization of transaction analytics, fraud alerts, risk distributions, and suspicious activity summaries for compliance analysts.

- **Suspicious Transaction Report (STR) Generation:** The system automatically generates structured Suspicious Transaction Reports for identified fraudulent transactions to support compliance and regulatory workflows.

- **Authentication and Role-Based Access Control:** Secure authentication and controlled access mechanisms are incorporated to manage user privileges and protect sensitive financial information within the monitoring platform.

User Characteristics

The proposed TBML detection framework supports multiple categories of institutional users involved in fraud monitoring, compliance verification, and regulatory supervision activities. Since the system processes sensitive transactional and compliance-related information, role-based access mechanisms are enforced to ensure secure and controlled interaction with the platform. The major user categories associated with the proposed framework are summarized as follows:

- **Bank Compliance Team:** The bank compliance team represents institutional analysts responsible for monitoring transactional activities and investigating suspicious financial behavior identified by the fraud-detection framework. These users are expected to possess domain knowledge related to Anti-Money Laundering (AML) procedures, transaction analysis, and institutional compliance operations. Authorized compliance personnel interact with analytical dashboards, review suspicious transaction records, verify fraud alerts, and manage institutional STR escalation workflows.

- **Regulatory Administrator:** The regulatory administrator functions as a supervisory authority responsible for overseeing fraud-monitoring activities across participating financial institutions. These users typically possess advanced regulatory and compliance expertise associated with financial investigation and institutional auditing operations. Regulatory administrators access consolidated Suspicious Transaction Reports (STRs), monitor institution-level compliance activities, and supervise large-scale suspicious transactional behavior across distributed banking environments.

Constraints

The development and deployment of the proposed TBML detection framework are subject to multiple operational, computational, and regulatory constraints associated with financial monitoring systems. These limitations influence the scalability, analytical performance, and real-time operational capabilities of the framework.

The major constraints associated with the proposed system are summarized as follows:

- **Data Privacy and Regulatory Constraints:** Financial institutions operate under strict privacy regulations and compliance policies that restrict direct sharing of sensitive transactional information across organizations.
- **Large-Scale Transactional Data Processing:** The system must process large volumes of interconnected financial transactions and graph relationships, which may increase computational complexity and memory utilization during real-time analysis.
- **Availability of Real-World TBML Datasets:** Access to publicly available Trade-Based Money Laundering datasets is limited due to confidentiality and regulatory restrictions, making large-scale real-world experimentation challenging.
- **Dynamic Fraud Patterns:** Money laundering strategies continuously evolve over time, requiring fraud detection models to adapt to changing transactional behaviors and emerging laundering techniques.
- **Infrastructure and Computational Limitations:** Training graph-based machine learning models and processing heterogeneous financial networks require significant computational resources, especially for large-scale transactional ecosystems.

3.1.2 Specific Requirements

The specific requirements of the proposed TBML detection framework define the operational functionalities, performance expectations, software dependencies, and computational resources required for efficient execution of the system. These requirements ensure that the framework can support real-time fraud analysis, scalable transaction monitoring, secure user interaction, and intelligent compliance management within financial institutions.

Functional Requirements

The functional requirements define the major operations and services that must be supported by the proposed TBML detection framework. The primary functional requirements of the system are summarized as follows:

- The system shall enforce role-based authorization policies to regulate operational access privileges for compliance analysts and regulatory administrators.
- The framework shall support ingestion and preprocessing of transactional datasets for heterogeneous financial graph construction.
- The proposed system shall model financial entities, transactions, and trade relationships as interconnected heterogeneous graph structures.
- The analytical engine shall perform graph-based fraud classification and generate transaction-level risk predictions using federated learning methodologies.
- The dashboard infrastructure shall support real-time visualization of suspicious transactions, fraud alerts, analytical summaries, and compliance-monitoring activities.
- The system shall enable institutional compliance teams to review suspicious transactions and perform fraud escalation or clearance procedures.
- The framework shall provide features to generate Suspicious Transaction Reports (STRs) for transactions identified as potentially fraudulent.
- The regulator-side monitoring interface shall support centralized supervision of institutional fraud reports and suspicious financial activities.
- The analytical interface shall provide transaction-filtering, search, and investigative querying functionalities for fraud-analysis workflows.

Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, and performance expectations associated with the proposed TBML detection framework. These requirements ensure that the system remains scalable, secure, reliable, and responsive while processing large-scale financial transactions and analytical workloads.

The primary non-functional requirements associated with the proposed system are summarized as follows:

- **Scalability:** The framework should support scalable processing of large transactional datasets and heterogeneous graph structures without significant degradation in analytical performance.
- **Real-Time Processing:** The system should support low-latency transaction analysis and fraud prediction to enable continuous real-time monitoring of suspicious financial activities.
- **Security and Privacy:** The framework should ensure secure access control, protected communication, and privacy-preserving handling of sensitive financial information and compliance-related data.
- **Reliability:** The system should maintain stable analytical performance and continuous monitoring capabilities during prolonged operational workloads and concurrent user access.
- **Availability:** The monitoring platform should provide consistent access to dashboards, fraud analytics, and reporting services for authorized institutional users.
- **Maintainability:** The system architecture should support modular development, efficient debugging, and simplified integration of additional analytical modules and monitoring functionalities.
- **Usability:** The dashboard interfaces and visualization systems should provide intuitive interaction mechanisms for compliance analysts and regulatory administrators during fraud investigation activities.

Software Requirements

The implementation of the proposed TBML detection framework requires multiple software technologies, machine learning libraries, backend services, database systems, and visualization frameworks to support graph-based fraud detection, federated learning, real-time transaction monitoring, and compliance management functionalities. The proposed framework integrates both analytical and monitoring components that require coordinated interaction between frontend dashboards, backend processing services, database

systems, and machine learning environments. These technologies collectively support scalable fraud analysis, secure transaction management, and efficient visualization of suspicious financial activities. The software requirements associated with the proposed system are summarized in Table 3.1.

Table 3.1: Software Requirements of the Proposed TBML Detection Framework

Category	**Technologies / Tools**	**Purpose in the System**
Operating System	Windows 11	Provides the development and deployment environment for backend services, databases, and analytical modules.
Programming Languages	Python, JavaScript	Used for backend development, machine learning workflows, API services, and frontend application development.
Frontend Frameworks	Next.js, React.js, Tailwind CSS	Supports responsive user interface development, dashboard visualization, and real-time monitoring functionalities.
Backend Framework	FastAPI	Facilitates asynchronous API communication between frontend dashboards and backend fraud detection services.
Database Systems	PostgreSQL, Memgraph	Stores transactional records, graph relationships, compliance information, and analytical data required for fraud analysis.
Machine Learning Frameworks	PyTorch, PyTorch Geometric	Supports deep learning operations, Graph Neural Network implementation, and graph-driven fraud analysis.
Federated Learning Framework	Flower	Enables distributed federated learning and privacy-preserving model aggregation across institutional environments.
Streaming Platform	Apache Kafka	Supports real-time transaction streaming and event-driven processing within the monitoring pipeline.
Containerization Platform	Docker	Provides containerized deployment and scalable execution environments for system integration.
LLM Integration Service	Groq LLM API	Supports automated generation of Suspicious Transaction Reports and compliance-oriented analytical summaries.

Hardware Requirements

The implementation of the proposed TBML detection framework requires the following hardware components to support graph processing, machine learning model execution, federated learning coordination, real-time transaction monitoring, and analytical dashboard visualization. The hardware requirements associated with the proposed system are summarized in Table 3.2.

Table 3.2: Hardware Requirements of the Proposed TBML Detection Framework

Hardware Component	**Minimum Requirement**	**Purpose in the System**
Processor	Intel Core i7 / AMD Ryzen 7	Supports efficient execution of graph analytical models, backend services, and machine learning computations.
RAM	16 GB	Supports graph datasets, real-time analytical workloads, and concurrent monitoring operations.
GPU	NVIDIA RTX 3050	Accelerates Graph Neural Network training and deep learning computations.
Storage	512 GB SSD	Stores transactional datasets, graph databases, analytical models, and generated compliance reports.

3.2 High level Design

This section describes the high-level design of the proposed TBML detection framework, including architectural strategies, software design considerations, and the overall system architecture that integrates graph-based fraud detection, federated learning, and real-time monitoring within a unified compliance platform.

3.2.1 Design Considerations

The design of the proposed TBML detection framework is influenced by multiple operational, analytical, and security-related factors associated with large-scale financial monitoring systems. Since Trade-Based Money Laundering activities involve interconnected financial entities, cross-border transactions, and hidden transactional dependencies, the system architecture must support scalable analytical processing, secure institutional collaboration, and efficient graph-based fraud analysis. The major design considerations

associated with the proposed framework are discussed in the following subsections.

General Considerations

The proposed TBML detection framework is designed to support large-scale transaction monitoring and fraud analysis across distributed financial environments. Since laundering activities typically involve interconnected accounts, trade entities, invoices, and transactional relationships, the framework utilizes graph-based modeling techniques to represent hidden financial dependencies more effectively than conventional relational systems. The architecture additionally supports real-time transaction processing and scalable analytical workflows for continuously increasing institutional transaction volumes.

Security, privacy preservation, and system scalability are important considerations within the proposed architecture due to the sensitive nature of financial and compliance-related information. To address these requirements, the framework incorporates secure authentication mechanisms, role-based access control, and federated learning strategies for protected inter-institutional collaboration without centralized transactional data sharing. The overall system is developed using a modular architecture to simplify integration, maintenance, and future enhancement of transaction monitoring, fraud detection, compliance reporting, and dashboard visualization services.

Development Methods

The proposed TBML detection framework is developed using a modular and iterative development methodology to support scalable system integration and continuous refinement of analytical functionalities. Individual system components including transaction monitoring services, graph construction pipelines, fraud detection models, federated learning environments, and dashboard interfaces are developed and tested independently before integration within the complete monitoring framework.

The development process additionally incorporates incremental model evaluation and experimental refinement to improve fraud detection performance and analytical reliability. Graph-based machine learning models are trained using processed transactional datasets, while frontend and backend services are integrated through API-based communication mechanisms to support real-time interaction between monitoring modules and analytical services.

Version control and collaborative development practices are utilized throughout the

implementation process to manage backend services, machine learning workflows, frontend interfaces, and analytical configurations efficiently. This modular development methodology simplifies debugging, testing, maintenance, and future enhancement of the proposed fraud detection framework.

3.2.2 Architectural Strategies

The architectural strategies adopted in the proposed TBML detection framework focus on supporting scalable financial transaction analysis, secure institutional collaboration, and efficient interaction between analytical services and monitoring interfaces. The framework integrates graph-based fraud detection models, backend analytical services, real-time dashboards, and distributed learning mechanisms within a unified monitoring environment. The major architectural strategies associated with the proposed framework are discussed in the following subsections.

Programming Language

The proposed TBML detection framework primarily utilizes Python for backend services, graph processing, machine learning workflows, and analytical computations. Python was selected due to its extensive support for deep learning libraries, graph analytical frameworks, and federated learning environments. Libraries such as PyTorch, PyTorch Geometric, and Flower are utilized for implementing graph neural network models and distributed learning mechanisms within the fraud detection framework.

The frontend monitoring dashboard is developed using Next.js and React.js to support responsive user interface development and real-time analytical visualization. JavaScript and TypeScript are utilized for implementing dashboard components, transaction monitoring interfaces, and compliance management services. FastAPI is additionally integrated to support API-based communication between frontend dashboards, backend services, and machine learning modules.

System Integration Architecture

The proposed framework follows a modular client-server and service-oriented integration architecture to support coordinated interaction between analytical modules, backend services, and frontend monitoring interfaces. The fraud detection models, transaction monitoring services, authentication modules, and compliance reporting components operate as independent services while maintaining synchronized communication through

backend APIs and shared database environments.

The machine learning modules responsible for graph processing and fraud prediction operate independently from the frontend monitoring dashboard. Processed fraud predictions, transaction classifications, and generated Suspicious Transaction Reports are synchronized through backend services to support real-time visualization and compliance management within the monitoring environment. This modular integration strategy simplifies scalability, maintenance, and future extension of the proposed framework.

Data Storage Management

The proposed TBML detection framework utilizes both relational and graph-based database systems to support efficient financial data management and analytical processing. PostgreSQL is utilized for storing structured transactional records, authentication information, compliance-related data, and generated Suspicious Transaction Reports. The relational database environment supports secure storage and efficient retrieval of operational monitoring information within the framework.

Memgraph is utilized as the graph database environment for storing interconnected financial entities and transactional relationships. Accounts, organizations, invoices, and transactions are represented as graph nodes and edges to support relational fraud analysis and graph neural network processing. The combined relational and graph-based storage strategy improves scalability, analytical flexibility, and efficient synchronization between backend analytical services and monitoring dashboards.

3.2.3 Overall System Architecture

The proposed TBML detection framework is designed to enable intelligent financial transaction monitoring, graph-based fraud analysis, federated learning, and automated compliance reporting across distributed institutional environments. It combines transaction processing, heterogeneous graph construction, machine learning-based fraud detection, and real-time monitoring to identify suspicious activities related to Trade-Based Money Laundering.

The architecture begins with multiple data sources, including banking systems, trade platforms, and external financial providers. Transaction records, trade documents, KYC information, and monitoring logs are collected and processed through a transaction processing module that performs data validation, cleansing, normalization, and event gener-

ation for analytical use.

The processed data is then converted into heterogeneous graph structures, where entities such as accounts, organizations, invoices, and transactions are represented as interconnected nodes and edges. This graph representation helps capture hidden financial relationships and suspicious transaction patterns required for graph-based learning.

The fraud detection layer consists of a Hybrid Graph Neural Network module for relational analysis and a temporal learning module for sequential transaction analysis. Together, these components learn complex laundering behaviors and improve fraud detection accuracy through combined risk prediction.

To ensure privacy and regulatory compliance, the framework adopts a federated learning approach where institutions train models locally without sharing sensitive financial data. Only model updates are transmitted to a global aggregation module, enabling collaborative learning while preserving data confidentiality.

The aggregated global model is used within the inference and risk scoring module to classify suspicious transactions and generate fraud risk scores. Transactions identified as high-risk are forwarded to the compliance management system for alert generation, Suspicious Transaction Report creation, and regulatory support. The complete architecture of the proposed TBML detection framework is illustrated in Figure 3.1.

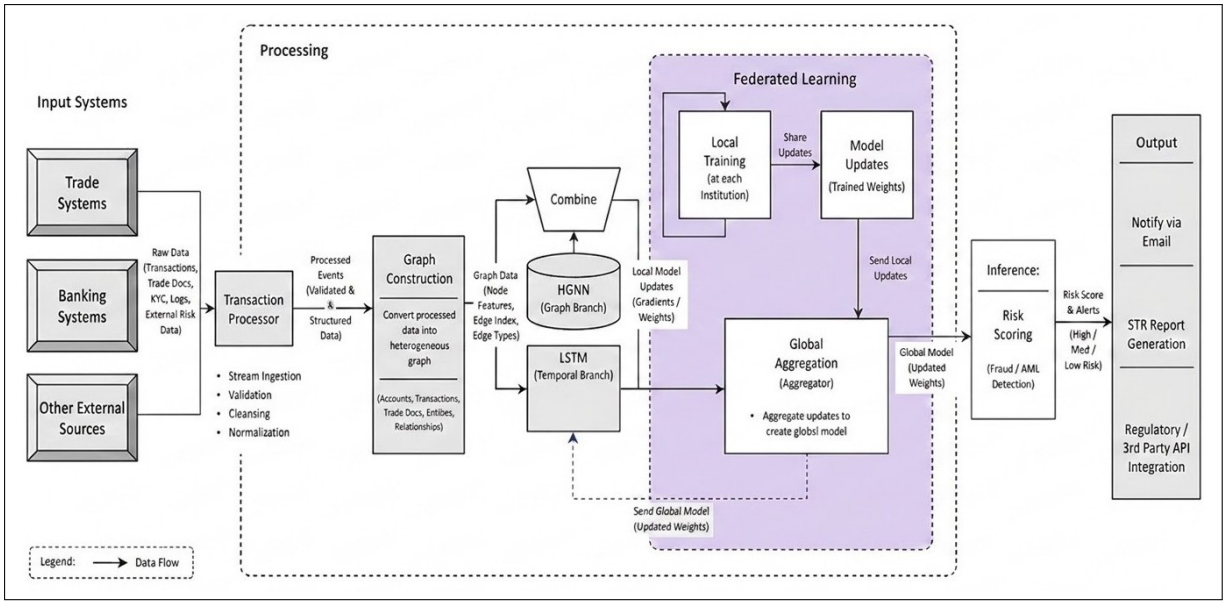

Figure 3.1: Overall Architecture of the Proposed TBML Detection Framework

3.2.4 Data Flow Diagrams

A Data Flow Diagram (DFD) is a graphical representation used to illustrate the movement of data within a system and the interactions between processes, external entities, and data repositories. DFDs provide a structured visualization of how information enters, is processed, stored, and transferred throughout a system architecture. In the proposed Federated Heterogeneous Graph Neural Network (HGNN)-Based Trade-Based Money Laundering (TBML) Detection System, DFDs are utilized to model the overall workflow of transaction acquisition, graph construction, federated learning, fraud analysis, and alert generation. The hierarchical decomposition of the system into Level 0, Level 1, and Level 2 diagrams enables a detailed understanding of the operational flow, subsystem interactions, and data processing mechanisms involved in real-time AML analytics. The diagrams presented in this section are designed using the Yourdon and De Marco DFD methodology to ensure standardized representation and structured system decomposition.

Data Flow Diagram Level 0

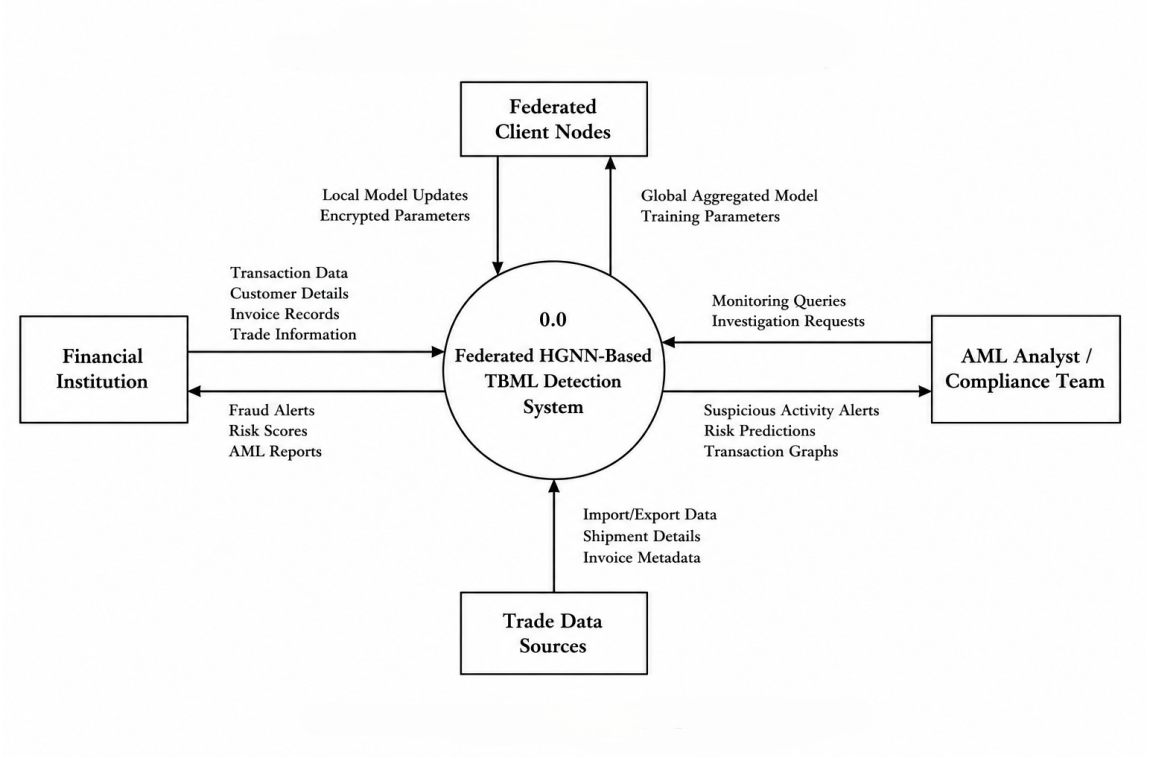

Figure 3.2: Level 0 Data Flow Diagram

Figure 3.2 illustrates the Level 0 Data Flow Diagram of the proposed Federated HGNN-Based TBML Detection System. The diagram provides a high-level contextual representation of the complete system as a single process interacting with external entities including Financial Institutions, Federated Client Nodes, AML Analysts, and Trade Data Sources. Financial institutions provide transaction records, customer information, invoice metadata, and trade-related details to the system for analysis. Trade data sources contribute import-export records and shipment information required for transaction validation and graph construction. Federated client nodes exchange encrypted local model updates and receive aggregated global model parameters during distributed training. The system processes these inputs to generate fraud alerts, risk predictions, AML reports, and suspicious activity notifications for compliance teams and financial analysts. The Level 0 DFD establishes the overall system boundary and illustrates the primary flow of information involved in distributed TBML detection and real-time financial fraud analytics.

Data Flow Diagram Level 1

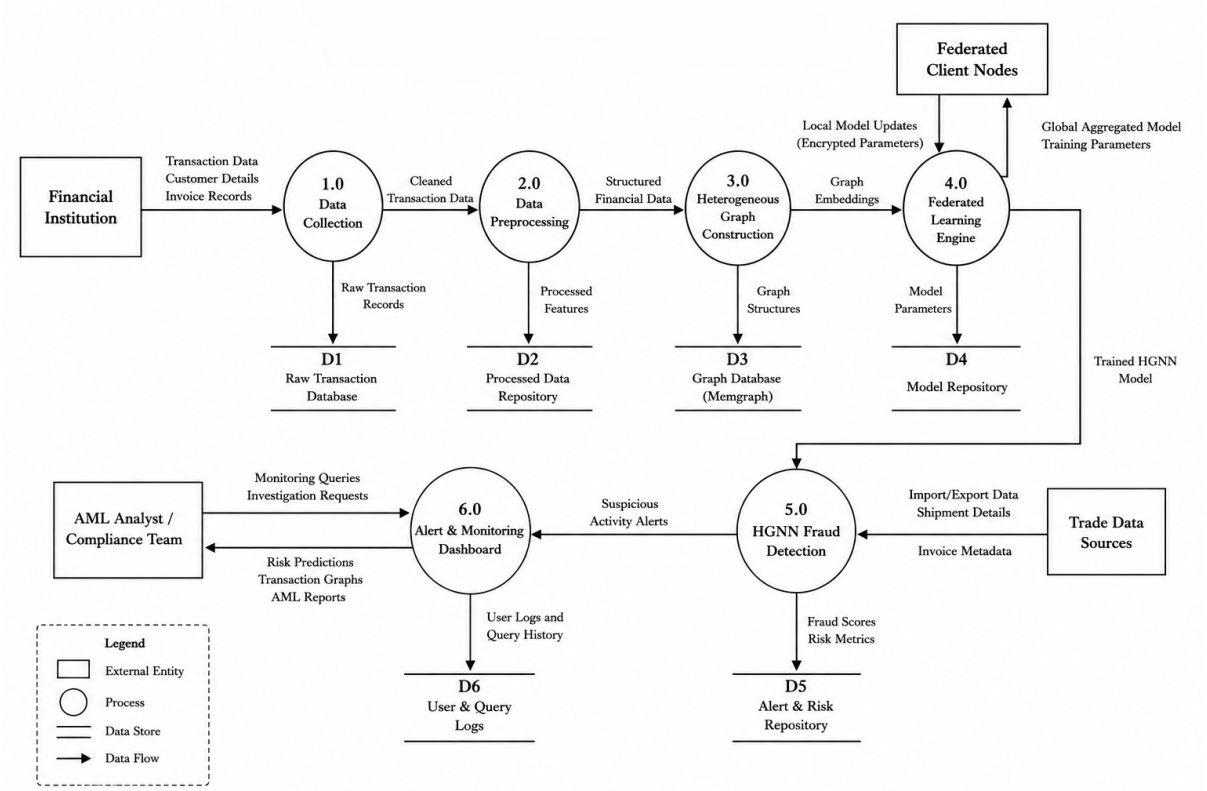

Figure 3.3: Level 1 Data Flow Diagram

Figure 3.3 presents the Level 1 Data Flow Diagram of the proposed TBML detection framework by decomposing the primary system into multiple functional processes. The

architecture consists of modules including Data Collection, Data Preprocessing, Heterogeneous Graph Construction, Federated Learning Engine, HGNN Fraud Detection, and Alert and Monitoring Dashboard. The Data Collection module acquires raw transactional records and customer information from financial institutions, which are subsequently cleaned and transformed within the preprocessing stage to generate structured financial features. These processed features are utilized by the graph construction engine to build heterogeneous graph representations stored within the graph database. The federated learning engine performs distributed collaborative model training using encrypted local updates received from federated client nodes while maintaining institutional data privacy. The trained HGNN model is then employed by the fraud detection engine to identify suspicious transactional patterns, calculate fraud scores, and generate anomaly alerts. Finally, the monitoring dashboard enables AML analysts and compliance teams to perform investigations, visualize transaction graphs, and access risk assessment reports for real-time anti-money laundering operations.

Data Flow Diagram Level 2

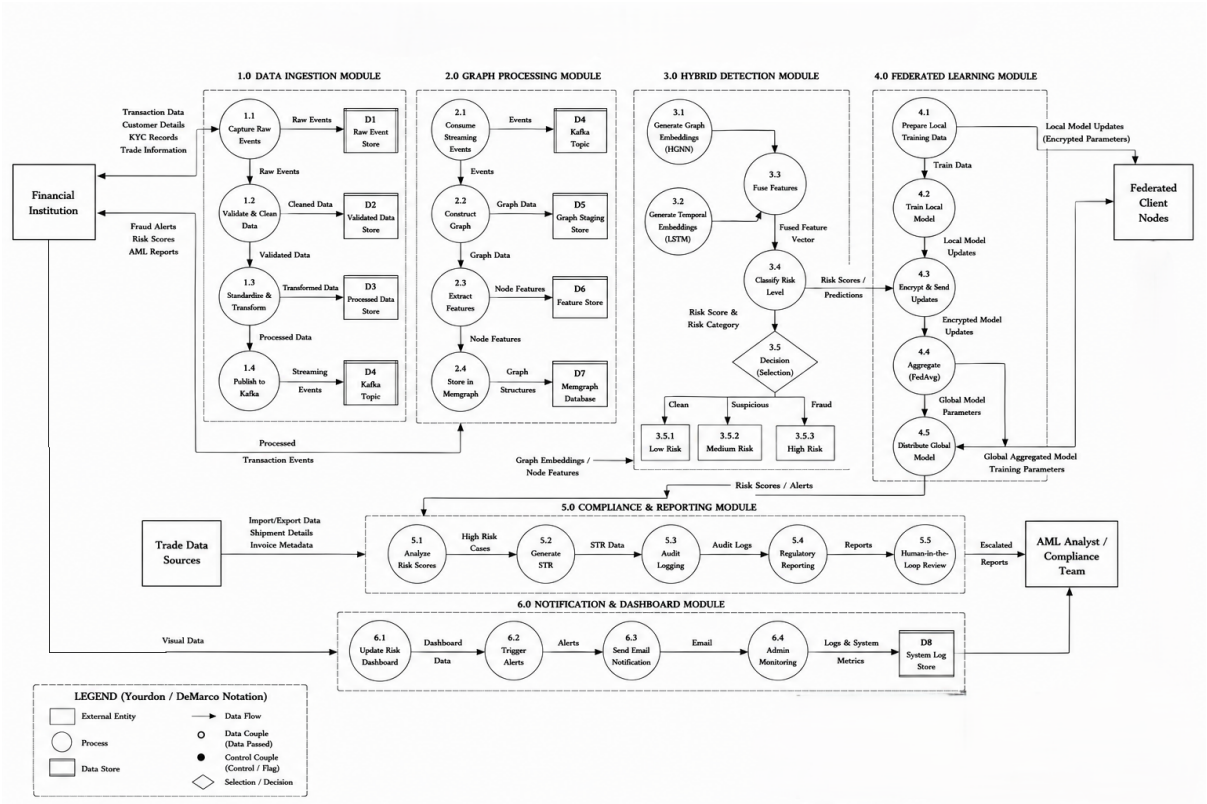

Figure 3.4: Level 2 Data Flow Diagram

Figure 3.4 illustrates the Level 2 Data Flow Diagram representing the internal decomposition of the HGNN Fraud Detection module within the proposed system. The process begins with Data Integration and Validation, where trade invoice records, shipment details, and transactional metadata are verified and filtered for inconsistencies. The validated information is then processed through Entity Resolution and Feature Extraction to identify relational attributes and generate contextual financial features required for graph-based learning. Subsequently, the Heterogeneous Graph Generation process constructs interconnected financial transaction graphs representing entities, accounts, invoices, and trade relationships. The HGNN Inference and Risk Scoring module applies graph neural network-based learning to calculate anomaly scores and identify suspicious transactional behavior. The resulting outputs are further processed through Anomaly Detection and Classification, followed by Alert Generation and Prioritization to rank suspicious activities according to calculated risk severity. Finally, the Result Aggregation and Reporting module generates consolidated fraud summaries, suspicious activity alerts, and analytical reports that are forwarded to the Alert and Monitoring Dashboard for investigation and compliance analysis. The Level 2 DFD provides a detailed representation of the internal analytical workflow employed by the proposed federated graph intelligence framework for real-time TBML detection.

3.3 Detailed System Design

This section provides a comprehensive overview of the detailed system design for the proposed TBML detection framework. It includes the structure chart illustrating the hierarchical organization of system components, functional descriptions of major modules, and the interaction workflow between different services and analytical components within the framework.

3.3.1 Structure Chart

The structure chart of the proposed TBML detection framework illustrates the hierarchical organization of the major functional modules operating within the system. The framework is divided into frontend, backend, fraud detection, federated learning, database, and compliance layers to support scalable financial monitoring and intelligent fraud analysis within distributed Anti-Money Laundering environments.

The frontend layer provides dashboard interfaces, transaction monitoring services,

risk visualization, and STR management functionalities for institutional users. Backend services coordinate API communication, transaction processing, authentication workflows, and access control operations between frontend interfaces and analytical modules. The fraud detection layer incorporates Hybrid HGNN–LSTM components responsible for graph-based analysis, temporal transaction modeling, feature fusion, and risk classification.

The federated learning layer enables privacy-preserving collaborative model training and global model synchronization between participating institutions. Database services manage transactional storage, graph relationships, audit logs, and compliance-related records, while the compliance layer supports STR generation, regulatory reporting, email notifications, and monitoring operations. The overall structural organization of the proposed TBML detection framework is illustrated in Figure 3.5.

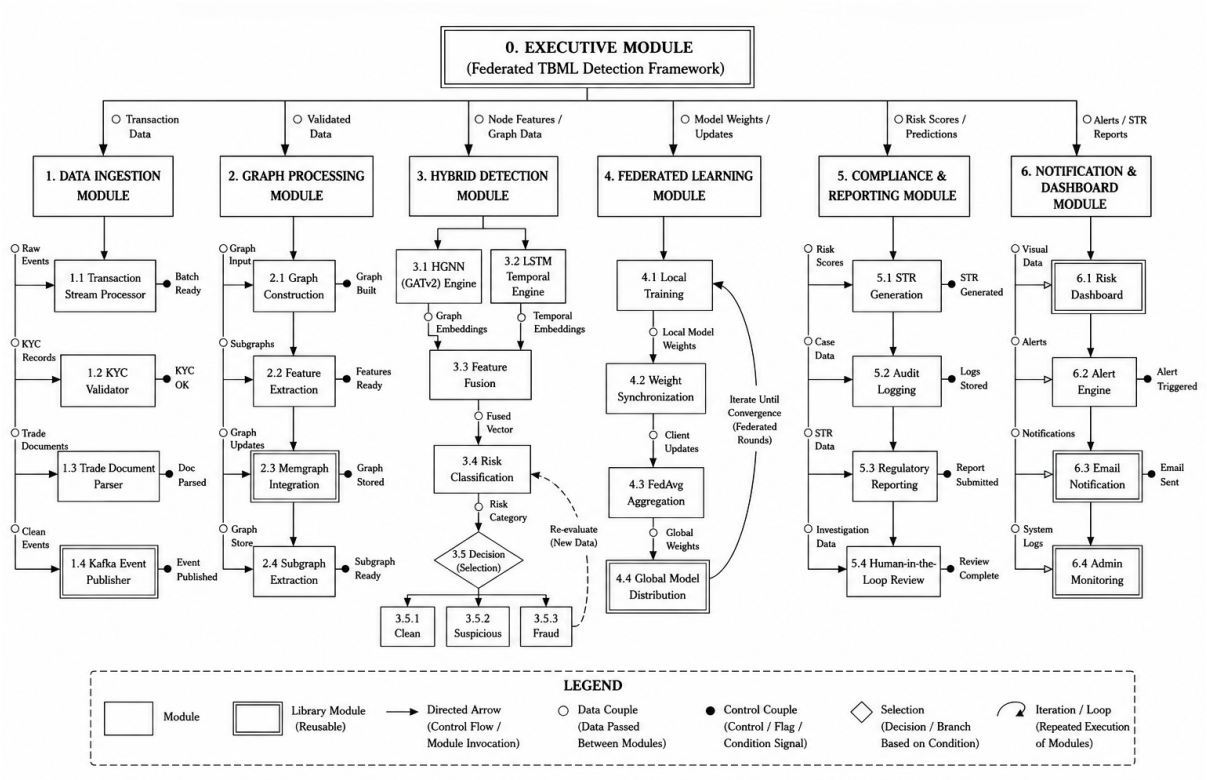

Figure 3.5: Structure Chart of the Proposed TBML Detection Framework

The structure chart illustrates the modular organization and interaction between analytical, monitoring, storage, and compliance-oriented services within the proposed framework. This modular design approach improves scalability, maintainability, service integration, and efficient coordination between frontend interfaces, backend analytical

services, and distributed fraud detection environments.

3.3.2 Functional Description of the Modules

The functional description of the modules explains the internal processing workflow and interaction between the major analytical, monitoring, and compliance-oriented components operating within the proposed TBML detection framework. The framework integrates transaction streaming services, graph construction mechanisms, hybrid HGNN–LSTM analytical models, institutional verification environments, and regulatory reporting systems to support intelligent Trade-Based Money Laundering detection and real-time compliance monitoring.

3.3.3 Module Interaction Workflow

The module interaction workflow illustrates the coordinated operational flow between transaction ingestion services, streaming environments, graph-based analytical modules, fraud classification mechanisms, institutional verification procedures, and regulatory monitoring systems functioning within the proposed framework. Financial transactions, KYC records, and trade documents are initially processed within the transaction processing layer and published into Kafka streaming topics for asynchronous event-driven communication between backend analytical services.

Kafka consumer services subsequently consume the transaction streams and forward the processed data to the graph construction and feature extraction module where heterogeneous financial transaction graphs are generated using accounts, transactions, trade entities, and document relationships stored within the Memgraph database. The extracted graph embeddings and relational features are processed by the Hybrid HGNN–LSTM fraud detection module to perform graph representation learning, temporal transaction analysis, and intelligent fraud prediction.

The generated fraud classifications and monitoring results are stored within the PostgreSQL database and visualized through the dashboard monitoring environment for institutional investigators and banking administrators. Suspicious transactions identified by the analytical system are reviewed by authorized bank administrators before being categorized as clean, suspicious, or confirmed fraudulent transactions.

Transactions verified as fraudulent are forwarded to the STR generation environment where Suspicious Transaction Reports are generated with analytical assistance from the

Groq API service. Generated STR reports are subsequently reviewed by regulatory authorities responsible for investigation management, inquiry procedures, and enforcement operations. Notification management services coordinate fraud alerts and institutional communication processes, while the Resend email service delivers notification emails and regulatory enforcement messages to the corresponding institutional authorities. The overall module interaction workflow of the proposed TBML detection framework is illustrated in Figure 3.6.

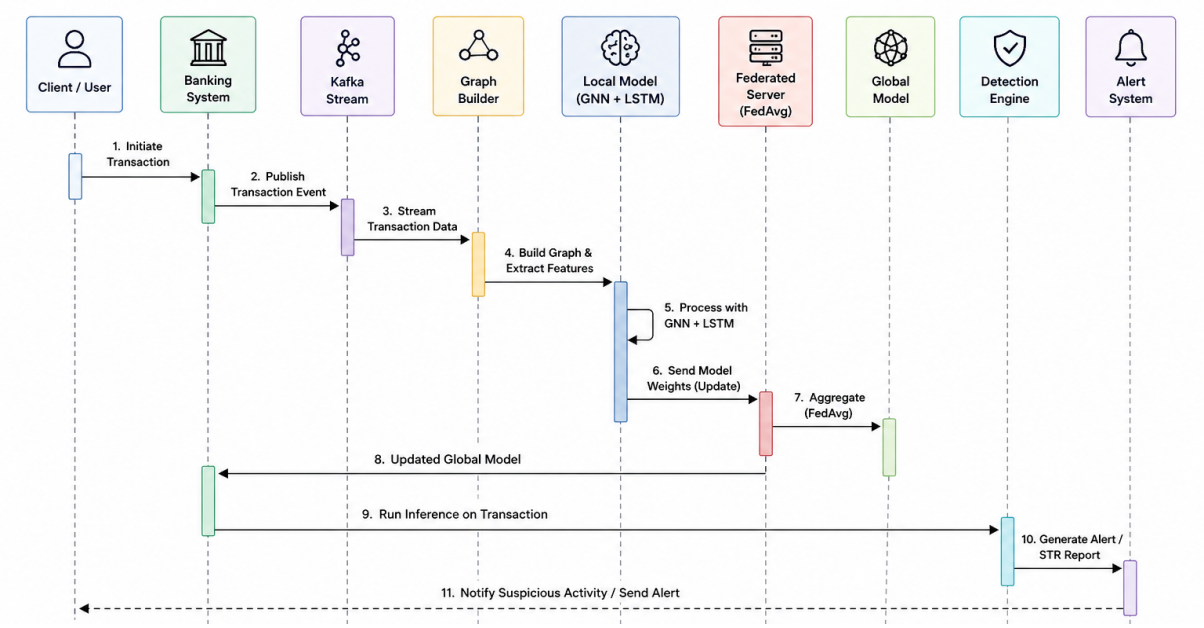

Figure 3.6: Workflow Interaction Diagram for the Proposed TBML Detection Framework

The workflow interaction diagram illustrates the coordinated interaction between streaming services, graph-based analytical modules, monitoring environments, institutional verification systems, and regulatory compliance services operating within the proposed framework. Such modular workflow integration supports scalable transaction processing, intelligent fraud analysis, compliance management, and real-time Anti-Money Laundering monitoring within distributed financial environments.

Transaction Processing Module

The transaction processing Module acts as the initial ingestion environment within the proposed TBML detection framework. This Module collects financial transactions, KYC records, trade documents, and institutional monitoring events originating from banking systems and external financial environments.

The input data processed within this Module includes sender and receiver account

identifiers, transaction amounts, commodity types, trade quantities, trade values, invoice details, transaction timestamps, and institutional monitoring attributes associated with financial activities. The collected transactional data is validated, normalized, and converted into structured transaction records to support reliable downstream analytical processing.

The processed transaction events are published into Kafka streaming topics to enable asynchronous communication between backend services and analytical modules. The generated transaction streams serve as the primary input for graph construction, feature extraction, and graph-based fraud detection operations within the proposed framework.

Graph Construction Module

The graph construction module transforms processed transaction streams into heterogeneous financial transaction graphs for downstream graph-based analytical processing. The module receives validated transaction events from Kafka consumer services and models financial interactions between accounts, transactions, and trade documents within a connected graph environment.

The generated graph schema consists of interconnected node entities including `(:Account)`, `(:Transaction)`, and `(:Document)` connected through relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` as illustrated in Figure 3.7. The heterogeneous graph structure preserves transactional connectivity, intermediary account interactions, and trade-document relationships required for identifying hidden financial dependencies associated with Trade-Based Money Laundering activities.

The module interacts with the Memgraph database to store graph structures, entity relationships, and transactional connectivity information required for downstream feature extraction and fraud analysis. The generated graph representations and extracted relational features are subsequently forwarded to the Hybrid HGNN–LSTM fraud detection module for intelligent transaction classification.

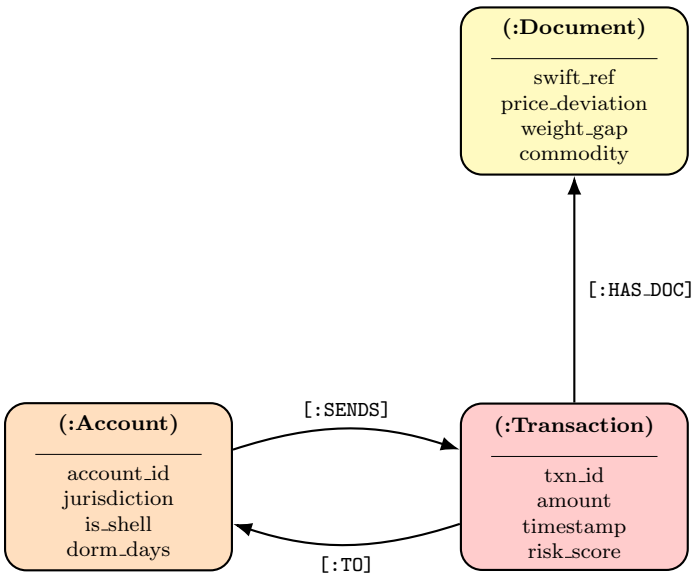

Figure 3.7: Heterogeneous graph schema representing entity attributes and transactional relationships in the proposed TBML detection framework.

Hybrid HGNN–LSTM Fraud Detection Module

The Hybrid HGNN–LSTM fraud detection module performs graph-based representation learning and temporal transaction analysis within the proposed TBML detection framework. The module receives KYC node features, graph edge relationships, SWIFT transaction attributes, sequential transaction tensors, and trade metadata extracted from the heterogeneous financial graph for downstream analytical processing.

The graph branch utilizes a three-layer GATv2 architecture to perform attention-based neighborhood aggregation and multi-hop message passing across interconnected financial entities. The stacked GATv2 layers improve two-hop neighborhood visibility and enable the model to capture hidden transactional relationships and intermediary account interactions associated with Trade-Based Money Laundering activities. The generated graph embeddings are globally pooled and fused with sequential transaction representations before being processed through the LSTM branch for temporal anomaly detection and transaction behavior analysis. In parallel, the trade branch processes trade-specific attributes including price deviation and weight gap scores using multilayer perceptron operations and feature normalization layers.

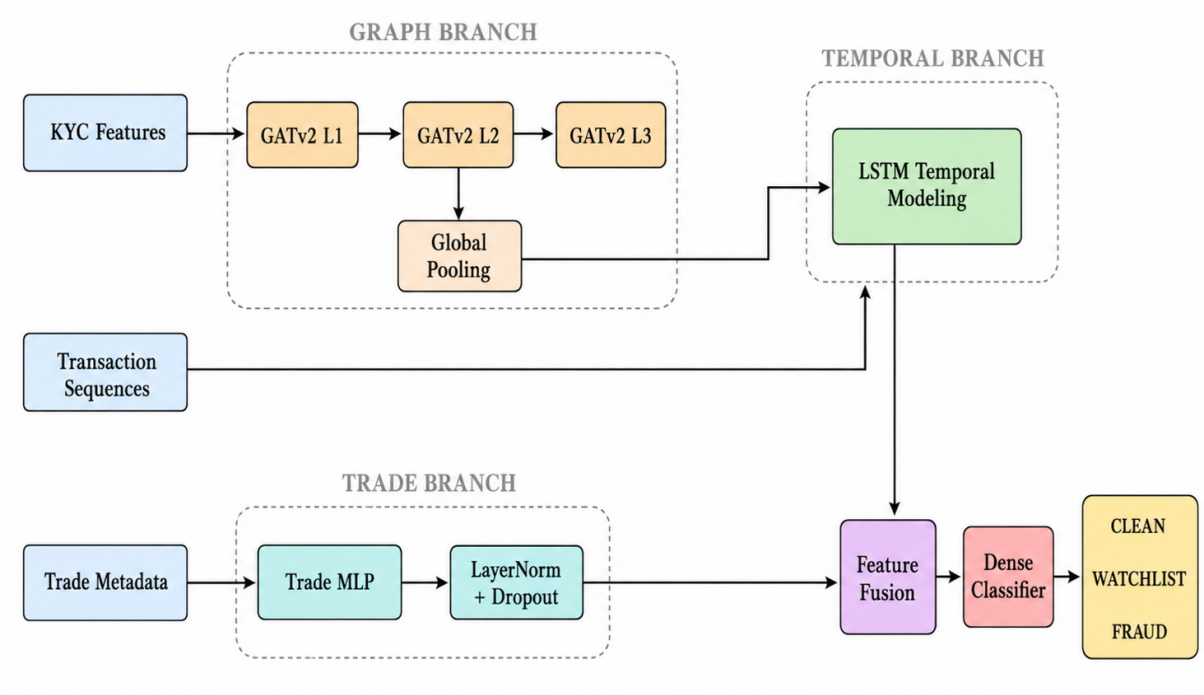

Figure 3.8: Architecture of the Proposed Hybrid HGNN–LSTM Fraud Detection Module

The generated graph embeddings, temporal representations, and trade feature vectors are combined within the feature fusion layer for downstream fraud prediction and transaction risk analysis. The final dense classification layer categorizes transactions into predefined classes including `CLEAN`, `WATCHLIST`, and `FRAUD`. The overall architecture of the proposed Hybrid HGNN–LSTM fraud detection module is illustrated in Figure 3.8.

Federated Learning Module

The federated learning module enables privacy-preserving collaborative fraud detection across distributed financial institutions without directly sharing sensitive transactional data. Each participating institution performs local model training using institutional transaction graphs, transaction sequences, and fraud classification labels generated within the analytical environment. Independent federated clients maintain isolated training environments and perform localized optimization of the Hybrid HGNN–LSTM model on institution-specific financial datasets.

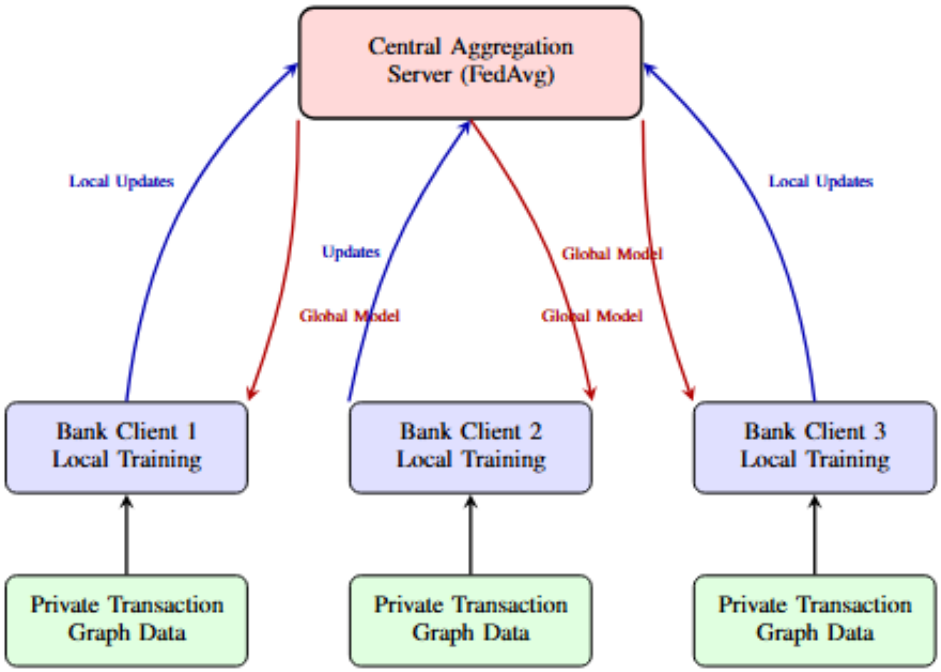

Figure 3.9: Federated learning architecture for privacy-preserving collaborative TBML detection across distributed institutions

The framework utilizes the Federated Averaging (`FedAvg`) strategy to aggregate distributed model parameters across participating institutions. During each federated communication round, locally trained model weights generated from individual institutional clients are transmitted to the central aggregation server where parameter averaging operations are performed to generate a globally optimized fraud detection model. The aggregated global model parameters are subsequently redistributed to participating institutions for synchronized fraud detection and continuous real-time analytical inference. The overall federated learning architecture of the proposed framework is illustrated in Figure 3.9.

STR Generation and Notification Module

The STR generation and notification module is responsible for compliance reporting and fraud alert management within the proposed TBML detection framework. Transactions verified as fraudulent by authorized banking administrators are forwarded to this module for Suspicious Transaction Report (STR) generation and regulatory processing. The module generates STR reports containing transaction details, fraud classifications, trade metadata, institutional information, and investigation summaries associated with

suspicious financial activities.

The Groq API service is utilized for analytical summarization and automated assistance during compliance documentation generation. Generated STR reports are subsequently forwarded to regulatory monitoring authorities for investigation and enforcement procedures. The notification management environment coordinates fraud alerts and institutional communication workflows, while the Resend email service delivers notification emails, investigation alerts, and regulatory enforcement messages to banking institutions and regulatory authorities.

Dashboard and Regulatory Monitoring Module

The dashboard and regulatory monitoring module provides real-time transaction visualization, fraud monitoring, and institutional investigation functionalities within the proposed TBML detection framework. The monitoring dashboard displays transaction classifications, fraud alerts, risk scores, STR records, and analytical monitoring information generated by the fraud detection environment. Separate dashboard environments are maintained for banking administrators and regulatory authorities to support institution-level monitoring and centralized compliance management operations.

Authorized banking administrators can view suspicious and fraudulent transactions identified by the analytical model and perform institutional verification procedures before categorizing transactions as `CLEAN`, `WATCHLIST`, or `FRAUD`. Suspicious transactions can either be escalated as fraudulent transactions or marked as clean after institutional verification procedures, thereby reducing false positive classifications within the monitoring environment. Transactions verified as fraudulent are subsequently forwarded for Suspicious Transaction Report (STR) generation and regulatory reporting.

Regulatory authorities monitor generated STR reports and institution-level fraud cases through centralized monitoring dashboards. The regulatory environment supports compliance monitoring, document verification requests, inquiry procedures, and account freeze operations associated with suspicious financial activities and the workflow is illustrated in Figure 3.10.

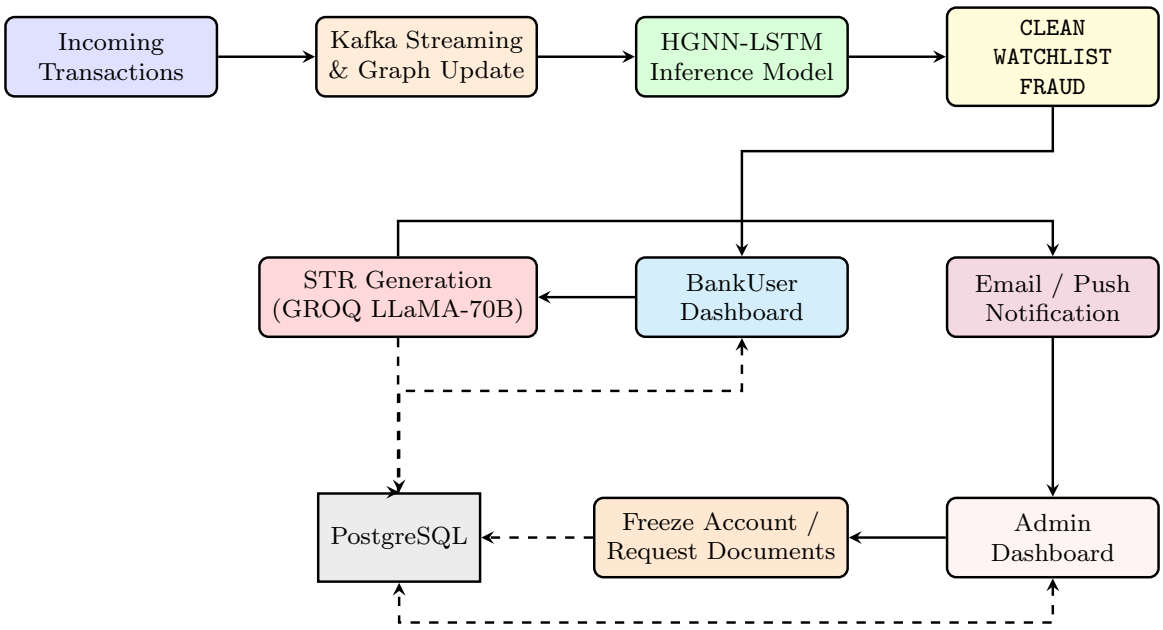

Figure 3.10: Real-time fraud inference and compliance workflow of the proposed TBML detection framework.

3.4 Summary

This chapter provided a comprehensive overview of the software specifications, architectural design, data flow diagrams, structure chart, and functional descriptions of the major modules operating within the proposed TBML detection framework. The detailed analysis of the system architecture, module interactions, and analytical workflows establishes a clear understanding of the operational mechanisms, service integration strategies, and real-time monitoring capabilities embedded within the framework to support intelligent Trade-Based Money Laundering detection and compliance management in distributed financial environments.

Chapter 4

Implementation and Testing of the Proposed TBML Detection Framework

CHAPTER 4

IMPLEMENTATION AND TESTING OF THE PROPOSED TBML DETECTION FRAMEWORK

This chapter details the implementation of the FedGAT-TBML framework. It covers real-time transaction ingestion and streaming, construction and storage of heterogeneous transaction graphs, a hybrid HGNN–LSTM detector for sequence-aware graph inference, federated learning orchestration for privacy-preserving model updates, and a web dashboard for compliance monitoring and STR generation. The implementation leverages Apache Kafka for durable streaming, Memgraph for graph storage and queries, PostgreSQL for audit and STR records, PyTorch Geometric for graph neural network components (e.g., GATv2Conv), Flower for federated orchestration, FastAPI for backend services, and Next.js/TypeScript for the frontend.

4.1 Implementation

This section provides a comprehensive overview of the implementation of the proposed TBML detection framework, detailing the programming languages, frameworks, platforms, code conventions, system architecture, and the strategies employed to address implementation challenges. The implementation integrates real-time transaction streaming, heterogeneous graph construction, hybrid analytical modeling, federated learning coordination, and compliance monitoring within a unified platform for effective Trade-Based Money Laundering detection and institutional oversight.

4.1.1 Programming Language and Framework Selection

The implementation primarily uses Python for backend services and model training (PyTorch + PyTorch Geometric — e.g., `GATv2Conv` for graph attention), Flower for federated orchestration (custom `FedAvg` strategy), and Apache Kafka for durable streaming. Frontend and UI services use Next.js, TypeScript and Tailwind; FastAPI provides REST endpoints. Memgraph stores heterogeneous transaction graphs and PostgreSQL holds audit logs and STR records. Services are containerized with Docker for reproducible deployment.

4.1.2 Platform Selection

The proposed TBML detection framework is developed within a Windows-based environment consisting of interconnected frontend monitoring services and backend analytical infrastructures. The frontend platform manages authentication workflows, dashboard visualization, STR generation, regulatory monitoring, and institutional transaction management, while the backend analytical environment performs transaction streaming, graph generation, fraud inference, and federated model synchronization operations.

The implementation utilizes Docker-based isolated runtime environments for Kafka services, graph databases, backend APIs, and federated learning modules to improve deployment consistency and distributed service orchestration. Memgraph is utilized for graph relationship management and transactional connectivity analysis, whereas PostgreSQL functions as the centralized synchronization database between analytical services and dashboard environments. Visual Studio Code and GitHub are utilized for collaborative development, source-code management, debugging, and version control operations throughout the implementation lifecycle.

4.1.3 Code Conventions

The codebase follows concise, practical conventions to improve readability and maintenance. Python modules and functions use `snake_case`; classes and UI components use `PascalCase`. Examples of graph entities and relationships include `(:Account)`, `(:Transaction)`, `(:Document)` and `[:SENDS]`, `[:TO]`, `[:HAS_DOC]`. REST endpoints follow clear, resource-oriented naming (e.g., `/api/transactions/{id}/score`). Configuration is managed via `.env` files, logging uses structured JSON logs, and type hints plus docstrings are used where helpful.

Naming Conventions

Naming conventions within the proposed framework are designed to remain semantically meaningful and aligned with domain-specific financial monitoring operations. Python variables, backend utility functions, Kafka services, and transaction-processing modules follow `snake_case` conventions, whereas analytical classes, federated learning modules, and frontend components utilize `PascalCase` notation. REST API endpoints, transaction services, and graph-processing utilities are named according to their operational functionality to improve implementation readability and maintainability.

Graph entities, transactional relationships, and database schemas additionally follow consistent naming conventions across graph and relational database environments. Examples include graph entities such as `(:Account)`, `(:Transaction)`, and `(:Document)`, along with transactional relationships including `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` utilized throughout graph construction and fraud analysis operations.

File Organization

The proposed TBML detection framework follows a modular project structure where frontend monitoring services and backend analytical environments are maintained as interconnected distributed applications. The frontend environment contains dashboard interfaces, authentication services, STR management workflows, API handlers, reusable UI components, and regulatory monitoring modules implemented using the Next.js App Router architecture and TypeScript-based service organization.

The backend analytical environment contains Kafka producers and consumers, graph construction services, Hybrid HGNN–LSTM model implementations, federated learning clients, inference pipelines, and database synchronization utilities responsible for real-time fraud analysis and transaction monitoring operations. Shared configuration files, utility services, logging environments, and centralized database connectors are additionally organized according to functional responsibilities to improve scalability, maintainability, and distributed service coordination across the proposed framework.

Environment Configuration and Logging

Environment variables and runtime configurations are securely managed using centralized `.env` configuration files to prevent direct exposure of API credentials, Kafka broker addresses, database configurations, authentication tokens, and federated learning parameters within the codebase. Backend services including Kafka consumers, Memgraph connectors, PostgreSQL clients, and fraud inference pipelines are initialized during runtime to support centralized service configuration and distributed analytical synchronization.

Logging mechanisms are integrated throughout transaction streaming services, graph-processing environments, federated learning modules, backend APIs, and fraud inference pipelines to support runtime monitoring, debugging, audit traceability, and compliance management operations. Fraud classifications, transaction verdicts, confidence scores, STR generation events, and institutional monitoring activities are additionally logged

within PostgreSQL environments for dashboard synchronization and regulatory investigation workflows.

4.1.4 Difficulties Encountered and Strategies Used to Tackle

Implementation challenges included noisy and imbalanced labels, large graph sizes and memory constraints, streaming latency, and privacy-preserving coordination across institutional clients. Mitigations applied in the implementation include class rebalancing and targeted data augmentation for scarce labels, subgraph sampling and mini-batching to control memory, Kafka buffering and asynchronous consumers to reduce end-to-end latency, and Flower with secure aggregation and containerized deployments (Docker) to isolate client environments while enabling federated training.

4.1.5 System Implementation

The system implementation of the proposed TBML detection framework follows a distributed event-driven architecture integrating Kafka-based transaction streaming, heterogeneous graph generation, Hybrid HGNN–LSTM analytical processing, federated collaborative learning, real-time fraud inference, and dashboard-driven compliance monitoring. Backend analytical services, graph databases, inference pipelines, and frontend monitoring environments communicate through synchronized APIs, Kafka streams, and centralized PostgreSQL audit infrastructures to support scalable real-time Trade-Based Money Laundering detection and institutional monitoring operations.

4.1.6 Methodology of Synthetic Dataset Synthesis

The generation of the synthetic dataset is engineered to emulate the multi-modal complexities inherent in international trade finance. The process initializes by constructing a baseline network of financial entities—Accounts, Documents, and Transactions—governed by power-law distributions to simulate realistic institutional transaction volumes. Subsequent stages facilitate the injection of complex temporal patterns, commodity-specific price anomalies, and multi-currency layering effects, creating a high-fidelity environment for model training.

The underlying implementation utilizes a modular pipeline that leverages the Pandas library for vectorized data manipulation and Memgraph for graph-structured persistence. Each transaction record is anchored to its corresponding trade documentation through a schema-aligned bridge, ensuring that the structural connectivity required for Graph Neu-

ral Network (GNN) feature extraction is preserved. By integrating temporal constraints and noise-injection modules, the pipeline creates a realistic representation of financial behaviors, ranging from standard legitimate interactions to camouflaged, high-velocity laundering topologies.

Adversarial Refinement for Dataset Fidelity

To improve the realism of the synthetic dataset and reduce unintended learning biases, the data-generation pipeline was refined using multiple adversarial and statistically grounded modifications [16]. These refinements were introduced to ensure that the fraud-detection framework learned meaningful transactional and structural behaviors instead of relying on simple heuristic shortcuts or deterministic patterns.

1. **Price Multiplier Dynamics:** Static pricing rules were replaced with Gaussian Mixture Models (GMMs) and stochastic blending strategies so that a portion of fraudulent trades closely resembled legitimate market pricing behavior, thereby reducing simple threshold-based detection.

2. **Elimination of Feature Bias:** The *port_log_status* attribute was removed to avoid direct label leakage and encourage the Graph Neural Network to learn relational and structural dependencies more holistically.

3. **KYC/Dormancy Engineering:** Overlapping dormancy periods and controlled noise injection were introduced to generate more realistic customer activity distributions rather than binary dormant/active identifiers.

4. **Universal Topology Injection:** Complex graph structures such as Cycles, Fan-In, and Gather-Scatter patterns were distributed across all transaction classes to prevent structural bias where certain graph shapes directly imply fraud.

5. **Refined Pattern Classification:** Explicit pattern tags such as LEGIT and SUSPICIOUS were removed, and a neutral *No_Pattern* category was introduced to separate structural topology from fraud labeling.

6. **Temporal Realism:** The dataset generation process was aligned to a realistic one-year operational timeline (Dec 2024–Dec 2025) with transaction durations modeled according to practical financial activity windows.

7. **Scale-Free Network Topology:** Pareto/Zipf-based node distributions were introduced to simulate high-volume institutional or "Whale" entities capable of masking suspicious transactional behavior within dense financial activity.

8. **Log-Normal Transaction Amounts:** Transaction values were generated using Log-Normal distributions to better reflect real-world financial transaction behavior instead of uniformly distributed synthetic amounts.

9. **Multi-Hop Structural Integrity:** Rule-based generation constraints were enforced across multi-hop transaction paths to maintain graph consistency and realistic financial relationship propagation.

10. **FX Settlement/Currency-Jump Awareness:** Multi-currency transactional flows and FX spread variations were incorporated to expose the framework to cross-border layering and TBML settlement patterns.

11. **Stochastic Trade Price Blending:** Illicit trade pricing distributions were blended with legitimate transactional behaviors to reduce detectable tabular shortcuts and encourage context-aware inference.

12. **Data Leakage Mitigation:** Auxiliary indicators capable of providing deterministic fraud identification were systematically removed to improve generalization during model training.

13. **Endurance-Based Simulation:** Multi-hop transactional cycles were modeled as sustained operational behaviors rather than isolated short-duration activities to improve structural realism within the generated financial graph.

Kafka-Based Transaction Streaming

The proposed TBML detection framework utilizes Apache Kafka to support asynchronous transaction streaming and event-driven communication between distributed analytical services. A Kafka producer continuously generates synthetic SWIFT transaction payloads containing sender identifiers, receiver identifiers, transaction amounts, commodity information, trade quantities, price deviation scores, and weight-gap attributes before publishing them into the `incoming_swift_messages` topic for downstream analytical processing.

Kafka consumer services continuously subscribe to the transaction stream and process incoming financial events in real time for graph generation and fraud inference operations. The consumer pipeline validates transaction payloads, constructs heterogeneous graph relationships within Memgraph, extracts multi-hop transactional subgraphs, and forwards graph tensors to the Hybrid HGNN–LSTM inference engine for downstream fraud prediction and PostgreSQL audit logging. The streaming architecture enables decoupled communication between frontend services, graph-processing environments, and distributed analytical modules operating within the proposed framework.

4.1.7 Graph Construction and Feature Extraction

The graph construction pipeline transforms streamed financial transactions into heterogeneous graph representations for downstream graph-based analytical processing. Incoming transaction payloads consumed from the `incoming_swift_messages` Kafka topic are processed using Cypher-based graph ingestion operations within Memgraph, where transactional entities and financial relationships are dynamically mapped into interconnected graph structures as illustrated in Figure 4.1.

The generated graph schema consists of heterogeneous node entities including `(:Account)`, `(:Transaction)`, and `(:Document)` connected through relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]`. The feature extraction pipeline subsequently performs constrained multi-hop subgraph traversal around the target transaction to generate node feature tensors, graph edge mappings, sequential transaction representations, and trade metadata tensors required for Hybrid HGNN–LSTM-based fraud inference operations.

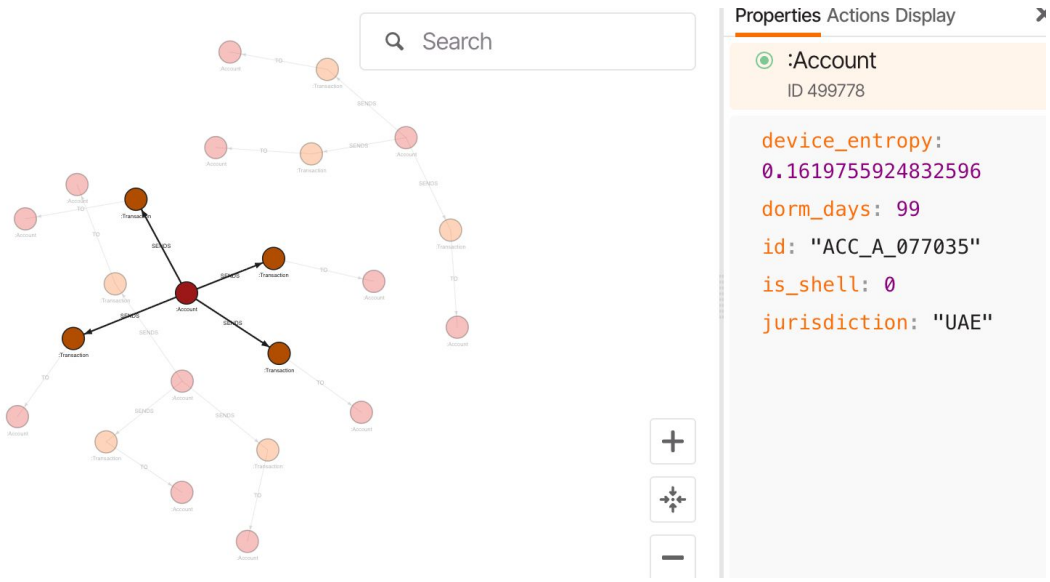

Figure 4.1: Heterogeneous financial transaction graph visualized within the Memgraph Lab environment.

Hybrid HGNN–LSTM Analytical Implementation

The proposed Hybrid HGNN–LSTM analytical module is implemented using PyTorch and PyTorch Geometric to perform graph-driven fraud analysis, temporal transaction modeling, and trade metadata learning for intelligent TBML detection. The implementation receives heterogeneous graph tensors generated from the Memgraph environment, including KYC node attributes, transactional edge relationships, sequential SWIFT transaction records, and trade-deviation features before forwarding them into the analytical inference pipeline. By combining graph attention learning with temporal sequence analysis and trade-feature fusion, the framework is capable of capturing both structural and behavioral fraud patterns across interconnected financial transaction networks.

The graph-learning branch utilizes a three-layer `GATv2Conv` architecture with multi-head attention mechanisms to perform neighborhood aggregation and graph message passing across interconnected financial entities [17]. The implemented graph layers process KYC-related node attributes such as shell-company indicators, jurisdiction metadata, and dormant-account characteristics to generate enriched graph embeddings capable of exposing hidden intermediary relationships and suspicious transaction-routing behavior. ReLU activation functions and dropout regularization are integrated after each graph convolution layer to improve representation stability during federated training operations. The generated graph embeddings are then aggregated using `global_mean_pool()` and fused with sequential SWIFT transaction features before being processed through the

LSTM branch for temporal anomaly detection and sequential transaction behavior modeling. The temporal branch captures repeated payment structures, abnormal transaction timing behavior, and cyclic financial movement patterns that are difficult to identify using standalone graph-learning approaches.

Trade-specific metadata such as price-deviation attributes and cargo weight-gap indicators are independently processed using multilayer perceptron operations consisting of fully connected layers, Layer Normalization, ReLU activations, and dropout regularization mechanisms. The generated graph embeddings, temporal sequence representations, and trade-feature vectors are fused within the final analytical layer before being forwarded into the dense classification module for fraud prediction. The final classification layer categorizes transactions into predefined classes including `CLEAN`, `WATCHLIST`, and `FRAUD`. Experimental evaluation demonstrated stable federated convergence and strong fraud-class Precision–Recall performance across distributed institutional datasets. The overall implementation workflow of the proposed Hybrid HGNN–LSTM analytical module is illustrated in Figure 4.2.

```
# ========================================
# HYBRID HGNN--LSTM IMPLEMENTATION
# ========================================

# --- Graph Attention Layers ---
# Multi-hop neighborhood aggregation
self.gat1 = GATv2Conv(
    in_channels=kyc_dim, out_channels=hidden_dim, heads=2, concat=False
)

self.gat2 = GATv2Conv(
    in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
)

self.gat3 = GATv2Conv(
    in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
)

# --- Temporal Sequence Branch ---
# Sequential SWIFT transaction modeling
self.lstm = nn.LSTM(
    input_size=hidden_dim + swift_dim, hidden_size=lstm_hidden, batch_first=True
)

# ========================================
# GRAPH MESSAGE PASSING
# ========================================

x = F.dropout(F.relu(self.gat1(kyc_x, edge_index)), p=0.2, training=self.training)

x = F.dropout(F.relu(self.gat2(x, edge_index)), p=0.2, training=self.training)

enriched_nodes = F.dropout(
    F.relu(self.gat3(x, edge_index)), p=0.2, training=self.training
)
```

```
# ========================================
# GLOBAL GRAPH POOLING
# ========================================

graph_context = global_mean_pool(enriched_nodes, batch).unsqueeze(1)

# ========================================
# GRAPH + TEMPORAL FEATURE FUSION
# ========================================

graph_context_expanded = graph_context.expand(seq_data.size(0), seq_data.size(1), -1)

combined_seq = torch.cat([seq_data, graph_context_expanded], dim=-1)

# Temporal anomaly detection
lstm_out, _ = self.lstm(combined_seq)

# ========================================
# TRADE FEATURE PROCESSING
# ========================================

trade_emb = self.trade_mlp(trade_features)

# ========================================
# FINAL FRAUD CLASSIFICATION
# ========================================

fused_vector = torch.cat([final_lstm_step, trade_emb], dim=1)

return self.classifier(fused_vector)
```

Figure 4.2: Implementation workflow of the proposed Hybrid HGNN–LSTM analytical module.

Federated Learning Implementation

The federated learning environment is implemented using the Flower federated learning framework to support distributed privacy-preserving fraud detection across multiple institutional clients. The implementation utilizes a centralized aggregation server and

three participating institutional clients, where each client performs localized training of the proposed Hybrid HGNN–LSTM analytical model using institution-specific transaction graphs, sequential SWIFT transaction representations, and fraud classification labels without directly sharing sensitive financial records outside the local analytical environment.

The centralized aggregation server implements a custom federated strategy by extending the `flwr.server.strategy.FedAvg` class and overriding the `aggregate_fit()` operation for distributed parameter aggregation. During each federated communication round, locally trained model weights generated from participating institutional clients are transmitted to the aggregation server where the `FedAvg` algorithm performs parameter averaging across synchronized client updates to generate a globally optimized fraud detection model. The server initializes the Hybrid HGNN–LSTM analytical architecture using synchronized feature dimensions including three-dimensional KYC node features, one-dimensional SWIFT transaction attributes, two-dimensional trade metadata features, hidden embedding dimensions of size 32, and three-class fraud prediction outputs.

The federated implementation performs ten synchronized communication rounds using complete client participation configurations including `fraction_fit=1.0`, `min_fit_clients=3`, and `min_available_clients=3` to maintain stable distributed convergence across institutional environments [18]. After the final aggregation round, the globally optimized model parameters are converted from Flower parameter streams into NumPy tensors using `parameters_to_ndarrays()` before being serialized and stored as `global_model.npz` for downstream real-time fraud inference operations. The implemented federated workflow enabled collaborative fraud learning, stable distributed convergence, and privacy-preserving analytical synchronization while maintaining complete institutional data isolation across participating financial organizations.

4.1.8 User Interface and Compliance Workflow Implementation

The user interface and compliance-monitoring environment is implemented using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and PostgreSQL to support secure institutional fraud monitoring and centralized regulatory investigation workflows. The frontend architecture follows a modular service-oriented design consisting of protected dashboard routes, middleware-based authentication guards, API handlers, Prisma-powered database interaction layers, and role-specific monitoring inter-

faces integrated with the federated analytical inference environment.

[Authentication and Role Validation Pipeline]**Authentication and Role Validation Pipeline**

The authentication workflow is implemented using JWT-based session authorization combined with centralized middleware validation and protected route interception mechanisms. Incoming requests targeting protected dashboard routes are validated through middleware guards where session tokens are verified against predefined authorization scopes including `BANK_ADMIN` and `REGULATOR`. Once validated, the session context is propagated across frontend environments through centralized authentication providers and Zustand-managed session states before protected dashboard components and compliance services are rendered within the client environment.

[Transaction Monitoring and Dashboard Rendering]**Transaction Monitoring and Dashboard Rendering**

Fraud predictions generated by the federated Hybrid HGNN–LSTM inference pipeline are synchronized into centralized PostgreSQL audit tables through backend transaction-processing services. Protected Next.js API handlers subsequently retrieve transaction classifications, fraud scores, STR metadata, and institutional monitoring information using Prisma query operations before dynamically rendering analytical outputs within role-specific dashboard interfaces. The banking administrator dashboard supports transaction filtering, fraud investigation workflows, watchlist escalation procedures, and false-positive reclassification operations through synchronized API-driven transaction state updates across centralized audit environments.

STR Generation and Notification Workflow

Transactions classified as fraudulent trigger Suspicious Transaction Report (STR) generation workflows implemented through secured API endpoints and PostgreSQL-backed audit synchronization services. Institutional administrators initiate STR generation requests through protected dashboard interfaces where transaction metadata, fraud classifications, investigation summaries, institutional remarks, and compliance status information are serialized and stored within centralized STR audit tables. The generated STR records are subsequently propagated into regulator-facing monitoring environments through synchronized notification services and backend compliance-routing workflows.

[Regulatory Investigation and Compliance Monitoring]**Regulatory Investigation**

and Compliance Monitoring

The regulatory monitoring environment consumes institution-generated STR records, fraud-classified transaction histories, compliance metadata, and supporting analytical evidence through protected API routes and Prisma-based database interaction layers. Regulatory authorities can perform investigation workflows including document-verification requests, institutional inquiry procedures, transaction review operations, and account-freeze initiation requests through centralized compliance interfaces synchronized with backend audit environments. The implemented monitoring workflow supports synchronized fraud prediction, institutional verification, STR lifecycle management, and centralized compliance monitoring within a unified real-time TBML analytical platform.

4.2 Testing and Validation

This section provides a comprehensive overview of the testing and validation processes employed to ensure the correctness, reliability, and performance of the proposed TBML detection framework. It includes descriptions of the testing environment, unit testing strategies for individual components, integration testing for end-to-end workflow validation, performance evaluation under simulated transaction loads, and security assessments to validate access control mechanisms and data protection measures within the framework.

4.2.1 Testing Environment

The testing process was conducted in a controlled development environment consisting of Python 3.11, PyTorch, PyTorch Geometric, Apache Kafka, Memgraph, PostgreSQL, and Flower federated learning services operating within isolated virtual environments created using `venv`. Frontend monitoring interfaces were implemented and validated using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and JWT-based authentication workflows.

The testing workflow additionally utilized Playwright-based browser automation for validating protected dashboard routes, institutional transaction workflows, STR generation pipelines, and compliance-monitoring interfaces. Database synchronization latency, transaction propagation consistency, backend API communication, and distributed fraud-inference workflows were validated under controlled institutional transaction loads to ensure reliable execution across distributed analytical environments.

4.2.2 Unit Testing

Each backend analytical service, dashboard workflow, authentication mechanism, and database synchronization component was independently tested to validate correctness, boundary handling, synchronization stability, and operational consistency before integration into the complete TBML detection framework.

Unit Testing of Authentication and RBAC Module

The authentication and role-based access control environment was independently validated to ensure secure user authentication, authorization management, middleware-driven route protection, credential verification, and institutional role isolation across banking and regulatory dashboard interfaces. The testing workflow additionally verified protected API access, invalid credential handling, unauthorized route restrictions, and role-specific dashboard rendering across distributed monitoring environments.

Table 4.1: TC-001: Unit Testing of Authentication and RBAC Module

Serial Number of Test Case	1.1
Name of Test Case	Protected Dashboard Access Validation
Feature being Tested	Authentication and Authorization Validation
Description	Validate protected route access, JWT verification, and role-based authorization
Sample Input	Invalid JWT session and unauthorized user request
Expected Output	Unauthorized access denied with authentication error response
Actual Output	Invalid sessions blocked and unauthorized users redirected successfully
Remarks	Successful test

Middleware-driven session validation successfully enforced protected route isolation and role-specific dashboard authorization across institutional monitoring environments. The testing workflow validated JWT token verification, session-expiration handling, credential validation, protected API access control, and middleware-based authorization checks across banking and regulator dashboards. Invalid credentials, unauthorized API requests, expired authentication sessions, and direct access attempts to protected routes were correctly rejected using backend access-control policies before protected resources

were rendered. The authentication workflow additionally verified successful role mapping for `BANK_USER` and `ADMIN` entities to ensure secure dashboard isolation and controlled access to institutional fraud-monitoring operations.

Unit Testing of Kafka Transaction Streaming

The Kafka transaction streaming environment was independently tested to validate asynchronous transaction publishing, real-time event synchronization, payload integrity validation, and downstream analytical communication across distributed backend services. The testing workflow additionally verified message delivery consistency, topic-based transaction propagation, malformed payload handling, and reliable synchronization between Kafka producer and consumer services operating within the proposed framework.

Table 4.2: TC-002: Unit Testing of Kafka Transaction Streaming

Serial Number of Test Case	1.2
Name of Test Case	Publish Valid SWIFT Transaction
Feature being Tested	Kafka Event Streaming and Message Synchronization
Description	Validate transaction publishing and topic-based event synchronization
Sample Input	JSON transaction payload containing sender, receiver, amount, and trade metadata
Expected Output	Transaction published into `incoming_swift_messages` topic successfully
Actual Output	Transaction streamed and synchronized successfully across Kafka services
Remarks	Successful test

The Kafka streaming module successfully validated asynchronous transaction publishing and backend event synchronization across distributed analytical services. Producer and consumer services maintained reliable message propagation and transaction consistency during continuous event-stream processing operations. Invalid payload structures, malformed transaction metadata, and incomplete financial events were correctly rejected through backend validation workflows before downstream graph-processing and fraud-inference operations were initiated.

Unit Testing of Graph Construction Module

The heterogeneous graph-construction environment was independently tested to validate transactional node generation, relationship mapping, multi-entity graph synchronization, and Cypher-based ingestion workflows within the Memgraph analytical environment. The testing workflow additionally verified graph consistency, entity-link generation, transaction propagation accuracy, and downstream graph availability for Hybrid HGNN–LSTM analytical inference operations.

Table 4.3: TC-003: Unit Testing of Graph Construction Module

Serial Number of Test Case	1.3
Name of Test Case	Heterogeneous Graph Entity Generation
Feature being Tested	Memgraph Graph Construction and Relationship Mapping
Description	Validate transactional node generation and graph relationship synchronization
Sample Input	Valid SWIFT transaction payload with account, trade, and document metadata
Expected Output	Graph nodes and transactional relationships created successfully within Memgraph
Actual Output	Heterogeneous graph entities and multi-hop relationships generated successfully
Remarks	Successful test

The graph-construction workflow successfully generated heterogeneous transactional graphs within the Memgraph environment using Cypher-based ingestion and relationship-mapping operations. Graph entities including `(:Account)`, `(:Transaction)`, and `(:Document)` were correctly synchronized with transactional relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` during continuous event-stream processing. The testing workflow additionally validated graph consistency, entity-link integrity, and multi-hop transactional connectivity required for downstream subgraph extraction and graph-based fraud inference operations within the proposed framework.

Unit Testing of STR Generation Workflow

The STR generation workflow was independently tested to validate suspicious transaction report generation, compliance-record synchronization, fraud-case escalation, and

regulator-side notification propagation across distributed institutional monitoring environments. The testing workflow additionally verified report persistence, transaction-metadata preservation, audit consistency, and downstream synchronization between banking dashboards and regulator-facing compliance interfaces.

Table 4.4: TC-004: Unit Testing of STR Generation Workflow

Serial Number of Test Case	1.4
Name of Test Case	Suspicious Transaction Report Generation
Feature being Tested	STR Generation and Compliance Synchronization
Description	Validate fraud escalation and regulator-side STR synchronization workflow
Sample Input	Fraud-classified transaction selected from institutional monitoring dashboard
Expected Output	STR record generated and synchronized into regulator compliance environment
Actual Output	STR generated, persisted, and synchronized successfully across monitoring services
Remarks	Successful test

The STR workflow successfully validated automated fraud-report generation, PostgreSQL audit synchronization, and regulator-side notification propagation across distributed institutional environments. Generated STR records correctly preserved transaction metadata, fraud classifications, institutional identifiers, and investigation references during downstream compliance synchronization operations. The testing workflow additionally verified successful dashboard synchronization, report persistence, and controlled propagation of suspicious transaction alerts between banking institutions and regulatory monitoring interfaces operating within the proposed framework.

Unit Testing of Notification Workflow

The notification synchronization environment was independently tested to validate fraud-alert propagation, institutional notification delivery, compliance-event synchronization, and regulator-side alert visibility across distributed monitoring interfaces. The testing workflow additionally verified notification persistence, dashboard update consistency, backend event propagation, and synchronization between banking and regulatory com-

pliance environments during fraud escalation operations.

Table 4.5: TC-005: Unit Testing of Notification Workflow

Serial Number of Test Case	1.5
Name of Test Case	Fraud Alert and Compliance Notification Propagation
Feature being Tested	Notification Synchronization and Alert Delivery
Description	Validate fraud-alert propagation across banking and regulator monitoring environments
Sample Input	Generated STR record linked to fraud-classified transaction
Expected Output	Compliance alerts synchronized into regulator-facing dashboard interfaces
Actual Output	Fraud notifications propagated and synchronized successfully
Remarks	Successful test

The notification workflow successfully validated fraud-alert propagation, institutional investigation updates, and STR lifecycle synchronization across distributed compliance-monitoring environments. Backend event-processing services maintained reliable notification delivery and dashboard synchronization during fraud escalation operations between banking institutions and regulatory authorities. The testing workflow additionally verified notification persistence, regulator-side alert visibility, and consistent compliance-state propagation across protected monitoring interfaces operating within the proposed framework.

4.2.3 Integration Testing

Integration testing was conducted to validate the interaction, synchronization consistency, and workflow stability between distributed analytical services, graph-processing environments, fraud inference pipelines, dashboard-rendering systems, and regulatory compliance workflows operating within the proposed TBML detection framework.

Integration Testing of Kafka and Memgraph Pipeline

This integration test validated end-to-end synchronization between Kafka-based transaction streaming services and the heterogeneous graph-construction environment oper-

ating within Memgraph. The testing workflow additionally verified event-stream consistency, transaction propagation reliability, graph-ingestion stability, and downstream synchronization between distributed backend services.

Table 4.6: TC-006: Integration Testing of Kafka and Memgraph Pipeline

Serial Number of Test Case	2.1
Name of Test Case	Transaction Stream and Graph Synchronization Workflow
Feature being Tested	Kafka Event Propagation and Memgraph Synchronization
Description	Validate transaction-stream synchronization and heterogeneous graph generation
Sample Input	Valid Kafka transaction payload with trade metadata and KYC attributes
Expected Output	Transactional graph entities and relationships generated successfully within Memgraph
Actual Output	Kafka events synchronized and heterogeneous graph generated successfully
Remarks	Successful test

The integration workflow successfully validated asynchronous transaction propagation between Kafka producer-consumer services and Memgraph graph-ingestion environments. Streamed SWIFT transaction events were reliably synchronized into graph-processing pipelines without transactional inconsistency or event-loss during continuous backend processing operations. The testing workflow additionally verified stable graph synchronization, multi-entity relationship generation, and downstream availability of transactional subgraphs required for graph-based analytical inference within the proposed framework.

Integration Testing of Fraud Prediction and Dashboard Synchronization

This integration test validated synchronization between fraud-prediction services, PostgreSQL audit environments, and role-based monitoring dashboards operating within the proposed TBML framework. The testing workflow additionally verified database persistence, dashboard synchronization consistency, and real-time rendering of fraud-classified transactions.

Table 4.7: TC-007: Integration Testing of Fraud Prediction and Dashboard Synchronization

Serial Number of Test Case	2.3
Name of Test Case	Fraud Prediction and Dashboard Rendering Workflow
Feature being Tested	PostgreSQL Synchronization and Dashboard Visualization
Description	Validate fraud-prediction synchronization into institutional monitoring dashboards
Sample Input	Fraud-classified transaction generated from analytical inference pipeline
Expected Output	Fraud transaction persisted and rendered successfully within dashboard interfaces
Actual Output	Dashboard synchronization completed successfully with valid fraud-state rendering
Remarks	Successful test

Fraud-prediction records, transaction metadata, and institutional risk classifications were successfully synchronized into centralized PostgreSQL audit environments before being rendered through protected dashboard interfaces. Dashboard synchronization workflows maintained consistent fraud-state propagation and real-time transaction visibility across distributed institutional monitoring services. The testing workflow additionally validated backend persistence consistency, dashboard update reliability, and secure rendering of fraud-monitoring data across role-specific compliance environments.

Integration Testing of STR Generation Workflow

This integration test validated the complete workflow between institutional fraud verification, STR generation services, regulator-side synchronization, and compliance-notification propagation operating within distributed monitoring environments. The testing workflow additionally verified fraud escalation consistency, report persistence, and synchronized regulator visibility across institutional compliance pipelines.

The STR synchronization workflow successfully validated fraud escalation, compliance-report generation, regulator-side dashboard synchronization, and distributed notification propagation across institutional monitoring environments. Generated STR records consistently preserved fraud classifications, investigation metadata, institutional identifiers,

and audit references during downstream compliance-processing operations. The testing workflow additionally verified successful propagation of compliance alerts and synchronized visibility of suspicious transaction reports across banking and regulatory monitoring interfaces.

Table 4.8: TC-008: Integration Testing of STR Generation Workflow

Serial Number of Test Case	2.4
Name of Test Case	Fraud Escalation and STR Compliance Workflow
Feature being Tested	STR Lifecycle Management and Compliance Synchronization
Description	Validate fraud escalation and regulator-side STR synchronization workflow
Sample Input	Fraud transaction escalated by institutional administrator
Expected Output	STR generated and synchronized successfully within regulator compliance dashboard
Actual Output	Fraud escalation and STR synchronization completed successfully
Remarks	Successful test

Integration Testing of Authentication and Protected Dashboard Rendering

This integration test validated synchronization between JWT-based authentication services, middleware-driven authorization guards, protected API routes, and role-specific dashboard-rendering environments operating within the proposed framework. The testing workflow additionally verified secure session propagation, authorization consistency, and protected resource accessibility across institutional monitoring interfaces.

Middleware-driven authentication workflows successfully validated session authorization, protected route synchronization, secure API accessibility, and institutional role isolation across distributed monitoring interfaces. Dashboard-rendering operations correctly enforced role-specific access-control policies for `BANK_USER`, `ADMIN`, and `REGULATOR` entities during backend synchronization workflows. The testing workflow additionally verified invalid-session rejection, unauthorized route restrictions, and secure propagation of authenticated dashboard resources across institutional compliance-monitoring environments.

Table 4.9: TC-009: Integration Testing of Authentication and Protected Dashboard Rendering

Serial Number of Test Case	2.5
Name of Test Case	Authenticated Dashboard Access and Rendering Workflow
Feature being Tested	Session Validation and Role-Based Dashboard Authorization
Description	Validate authenticated dashboard rendering and protected route authorization
Sample Input	Valid JWT-authenticated institutional user session
Expected Output	Authorized dashboard rendered successfully based on institutional role
Actual Output	Protected dashboard rendered successfully with valid role authorization
Remarks	Successful test

4.2.4 System Testing

System testing was conducted to validate the operational stability, concurrent transaction-handling capability, database synchronization performance, and workflow-level response consistency of the proposed TBML detection framework under distributed institutional monitoring environments. The testing workflow focused on browser-based end-to-end validation, protected dashboard execution, concurrent PostgreSQL scalability analysis, API response-time benchmarking, and operational latency evaluation across fraud-monitoring and compliance-management services.

Browser Automation and Workflow Validation

Browser-based end-to-end validation was performed using Playwright automation workflows executed within Chromium testing environments. The testing workflow validated protected dashboard rendering, role-based authentication, STR lifecycle execution, fraud-notification propagation, compliance-action synchronization, and institutional transaction visibility restrictions across distributed frontend and backend services. All automated workflow executions completed successfully without unauthorized access, synchronization failure, or dashboard inconsistency during operational validation.

Table 4.10: TC-010: Browser Automation and Workflow Validation Results

Workflow Validation	**Status**
Admin Authentication and Dashboard Rendering	Successful
Fraud Case Retrieval and STR Visibility	Successful
GET_DOCUMENTS Compliance Workflow	Successful
FREEZE_ACCOUNT Regulatory Workflow	Successful
Fraud Verdict Marking Workflow	Successful
Bank User STR Generation Workflow	Successful
Fraud Notification Propagation Workflow	Successful
Protected Route and RBAC Validation	Successful
Institutional Transaction Visibility Restriction	Successful
Sign-In Page Smoke Testing	Successful

The Playwright automation workflow successfully validated sequential execution of institutional fraud-monitoring operations across protected dashboard environments. Browser automation additionally verified secure session persistence, API synchronization consistency, workflow-level authorization enforcement, and stable dashboard rendering across `ADMIN` and `BANK_USER` operational environments. The testing workflow further confirmed successful STR generation, fraud escalation, regulator-side compliance synchronization, and distributed notification propagation across automated end-to-end execution workflows.

Concurrent Database Scalability and API Performance Analysis

Concurrent database scalability testing was conducted to evaluate PostgreSQL synchronization stability, dashboard query performance, fraud-filtering latency, and STR retrieval response consistency under increasing institutional user loads. The benchmarking workflow was implemented using Prisma ORM and Next.js backend services, where each simulated institutional user executed five continuous transactional requests during concurrent dashboard-access operations. Controlled concurrency execution was implemented using batched asynchronous request propagation with pooled PostgreSQL

database connections and a maximum limit of 15 active concurrent connections to regulate transactional synchronization, prevent backend resource exhaustion, and maintain stable query-execution behavior during high-volume monitoring workloads.

Latency benchmarking was performed by capturing request-initiation and response-completion timestamps during concurrent API execution workflows. For each transactional request, the benchmarking workflow first recorded the request initiation timestamp using `startTime = performance.now()`, followed by asynchronous API request execution and backend-response synchronization. Upon successful completion of the transactional request, the response-completion timestamp was recorded using `endTime = performance.now()`, and the operational latency was computed using `latency = endTime - startTime`. Concurrent workload execution was implemented using parallel asynchronous request propagation, where multiple simulated institutional users executed simultaneous dashboard and compliance-query operations under controlled concurrency constraints.

Average latency measurements represented the mean response time observed across all successful transactional requests, while P95 latency measurements captured upper-bound response behavior experienced by 95% of requests during peak synchronization conditions. The benchmarking workflow additionally evaluated request throughput, API synchronization consistency, transaction persistence reliability, and stable PostgreSQL query execution across distributed compliance-monitoring environments.

The benchmarking metrics were computed using the following operational latency equations:

$$L_{\text{avg}} = \frac{1}{N} \sum_{i=1}^{N} (T_{\text{response}_i} - T_{\text{request}_i}) \tag{4.1}$$

$$L_{95} = \text{Percentile}_{95}(L_1, L_2, \ldots, L_N) \tag{4.2}$$

Table 4.11: TC-011: Dashboard Transaction Retrieval Scalability Analysis

Concurrent Users	**Total Requests**	**Avg Latency (ms)**	**P95 Latency (ms)**	**Success Rate**
10	50	297.33	884.28	100%
50	250	156.05	174.75	100%
100	500	159.99	181.30	100%
250	1250	158.53	176.99	100%
500	2500	162.14	180.72	100%

The dashboard transaction retrieval workflow demonstrated stable query scalability and consistent dashboard-rendering performance across increasing institutional user loads. Average latency values remained nearly constant between 50 and 500 concurrent users, indicating efficient PostgreSQL query execution, optimized transactional indexing, and stable backend synchronization during high-frequency dashboard-access operations. The relatively low variation between average and P95 latency measurements further indicated predictable response behavior and controlled query-execution overhead under concurrent transactional workloads.

Table 4.12: TC-012: Fraud Transaction Filtering Scalability Analysis

Concurrent Users	**Total Requests**	**Avg Latency (ms)**	**P95 Latency (ms)**	**Success Rate**
10	50	159.58	174.04	100%
50	250	158.15	181.58	100%
100	500	147.56	170.37	100%
250	1250	148.18	164.58	100%
500	2500	263.80	731.88	100%

The fraud-transaction filtering workflow demonstrated efficient analytical-query execution and stable institution-specific transaction retrieval during concurrent dashboard operations. Lower average latency values under moderate workloads indicated optimized fraud-state filtering and indexed transactional query execution across backend analytical services. Increased P95 latency under 500-user workloads indicated temporary synchronization overhead during peak concurrent analytical filtering operations; however, the framework maintained complete request reliability, stable API synchronization, and consistent fraud-state propagation throughout distributed monitoring workflows.

Table 4.13: TC-013: STR Compliance Report Retrieval Scalability Analysis

Concurrent Users	**Total Requests**	**Avg Latency (ms)**	**P95 Latency (ms)**	**Success Rate**
10	50	2694.21	7311.50	100%
50	250	2101.61	4791.52	100%
100	500	856.39	2149.27	100%
250	1250	467.03	600.67	100%
500	2500	465.62	596.76	100%

The STR compliance retrieval workflow exhibited comparatively higher latency due to compliance-report aggregation, transaction-history synchronization, regulator-side metadata retrieval, and multi-table relational query execution during report-generation operations. However, the gradual reduction in average latency under increasing concurrent workloads indicated improved PostgreSQL query stabilization, efficient pooled connection utilization, and optimized relational-query execution during sustained processing operations.

Despite the additional computational overhead associated with compliance-report generation, the framework maintained stable request consistency and complete request success rates across distributed monitoring environments. The testing workflows validated reliable backend synchronization, controlled API response behavior, stable federated analytical execution, and consistent PostgreSQL query performance under increasing institutional workloads.

The conducted testing workflows validated the synchronization reliability, operational stability, and execution consistency of the proposed TBML detection framework across distributed monitoring environments. The testing results confirmed stable backend communication, reliable transactional synchronization, controlled API response behavior, and successful workflow execution across graph-processing, federated analytical inference, dashboard synchronization, and compliance-management services. Concurrent scalability validation additionally demonstrated stable PostgreSQL query execution and controlled latency characteristics under increasing institutional workloads.

4.2.5 Model Performance Evaluation

The proposed federated Hybrid HGNN–LSTM framework was evaluated across three distributed institutional clients (BANKA, BANKB, and BANKC) to validate fraud-classification capability and federated convergence stability across heterogeneous transactional environments. Multi-class prediction performance for *Clean*, *Watchlist*, and *Fraud* transaction states was analyzed using Precision, Recall, F1-Score, confusion-matrix validation, and Precision–Recall benchmarking. Federated parameter synchronization was performed using the FedAvg aggregation strategy across 10 federated communication rounds.

The transactional dataset was partitioned using a stratified temporal split strategy to preserve chronological separation between training and testing samples while maintaining balanced representation across minority fraud categories. Weighted cross-entropy optimization and gradient clipping mechanisms were integrated to stabilize distributed training and address fraud-class imbalance during federated convergence operations. Precision–Recall evaluation was prioritized over standalone classification accuracy due to the highly imbalanced nature of illicit transactional behavior within institutional financial environments.

Multi-Class Classification Performance

The final federated classification results obtained after 10 federated aggregation rounds across all participating institutional environments are summarized in Table 4.14. The evaluation was conducted to analyze the framework's ability to distinguish legitimate, suspicious, and fraudulent transactional behaviors across distributed institutional datasets under federated learning conditions. The obtained results additionally validated stable analytical convergence and consistent fraud-classification performance across all participating banking clients.

The evaluation results demonstrated stable multi-class classification performance across all distributed institutional clients. Clean transactional states achieved near-perfect Precision and Recall, thereby minimizing false-positive propagation during large-scale monitoring operations. The Watchlist class maintained Recall values above 0.97, indicating effective identification of suspicious transactional behavior requiring institutional review workflows.

Table 4.14: Federated Multi-Class Classification Performance result

Institution	Transaction Class	Precision	Recall	F1-Score
BANKA	Clean (0)	1.00	1.00	1.00
	Watchlist (1)	0.79	0.97	0.87
	Fraud (2)	0.96	0.73	0.83
BANKB	Clean (0)	1.00	1.00	1.00
	Watchlist (1)	0.79	0.99	0.88
	Fraud (2)	0.98	0.73	0.84
BANKC	Clean (0)	1.00	1.00	1.00
	Watchlist (1)	0.79	0.99	0.88
	Fraud (2)	0.98	0.73	0.84

Fraud-class predictions achieved Precision values between 0.96 and 0.98, validating reliable identification of high-risk transactional activity with limited false fraud escalation. Although Fraud Recall stabilized near 0.73, the framework preserved strong fraud-discrimination capability despite highly imbalanced transactional distributions and complex multi-hop laundering structures. The overall results additionally validated stable federated generalization across distributed institutional environments without requiring centralized sharing of sensitive financial datasets.

Federated Convergence Analysis

The federated convergence workflow was evaluated using longitudinal Accuracy, Precision, and Recall progression analysis across iterative federated aggregation rounds. The convergence curves were generated independently for all participating institutional clients to validate distributed synchronization consistency and global-weight stabilization during federated optimization workflows.

BANKA Federated Convergence

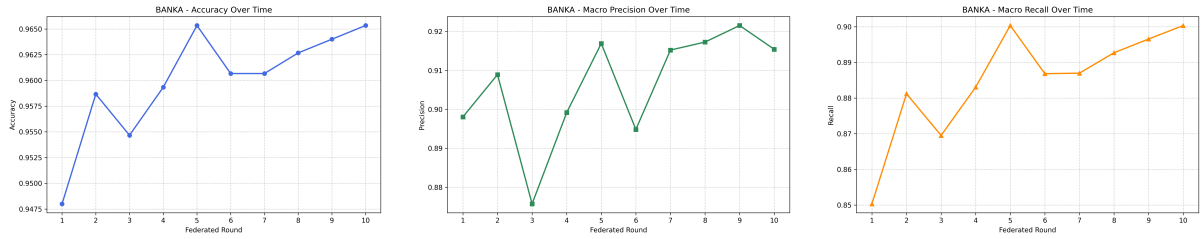

Figure 4.3: BANKA Federated Accuracy, Precision, and Recall Progression Across Federated Aggregation Rounds

As illustrated in Figure 4.3, the BANKA institutional client demonstrated progressive federated convergence stability across iterative aggregation rounds. Initial fluctuations observed during early federated synchronization stages were progressively stabilized through global parameter aggregation and distributed weight synchronization. The Accuracy, Precision, and Recall progression curves demonstrated reduced oscillatory variance during later communication rounds, thereby validating effective federated coordination and improved analytical consistency across institutional training environments.

BANKB Federated Convergence

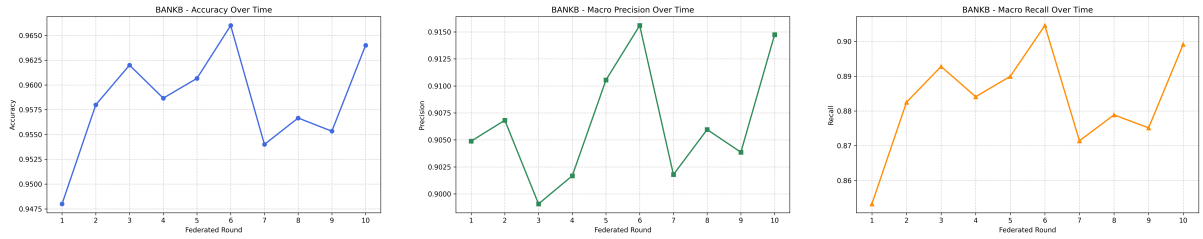

Figure 4.4: BANKB Federated Accuracy, Precision, and Recall Progression Across Federated Aggregation Rounds

As illustrated in Figure 4.4, the BANKB institutional client demonstrated stable convergence characteristics throughout iterative federated aggregation workflows. Distributed parameter synchronization progressively improved analytical stability across successive communication rounds and reduced localized classification variance during later training stages. The stabilized progression behavior additionally validated successful cross-bank parameter sharing and consistent distributed learning performance during federated optimization workflows.

BANKC Federated Convergence

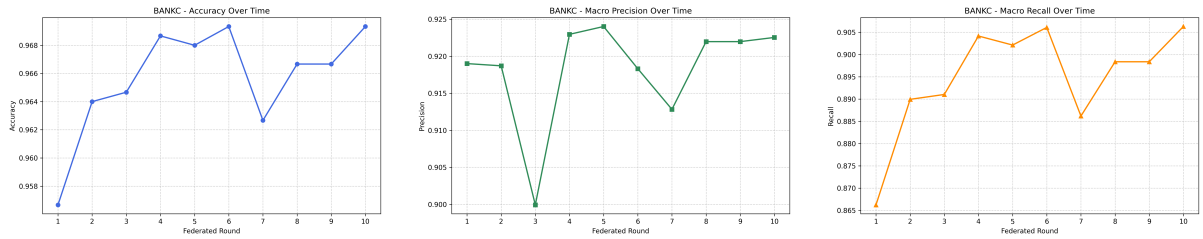

Figure 4.5: BANKC Federated Accuracy, Precision, and Recall Progression Across Federated Aggregation Rounds

As illustrated in Figure 4.5, the BANKC institutional client demonstrated comparatively stable federated convergence behavior across all communication rounds. The

Accuracy, Precision, and Recall metrics progressively converged with minimal variance during later aggregation stages, thereby validating efficient federated synchronization and stable distributed optimization behavior. The convergence characteristics additionally confirmed successful global-model generalization across heterogeneous institutional transaction environments.

Precision–Recall and Confusion Matrix Validation

Precision–Recall analytical validation and confusion-matrix evaluation were performed to analyze fraud-discrimination capability, minority-class detection behavior, and false-positive propagation across distributed institutional environments. Since fraudulent transactional behavior represented a significantly smaller proportion of the overall institutional transaction distribution, Precision–Recall evaluation provided a more operationally reliable assessment than standalone classification accuracy metrics.

BANKA Precision–Recall and Confusion Matrix Validation

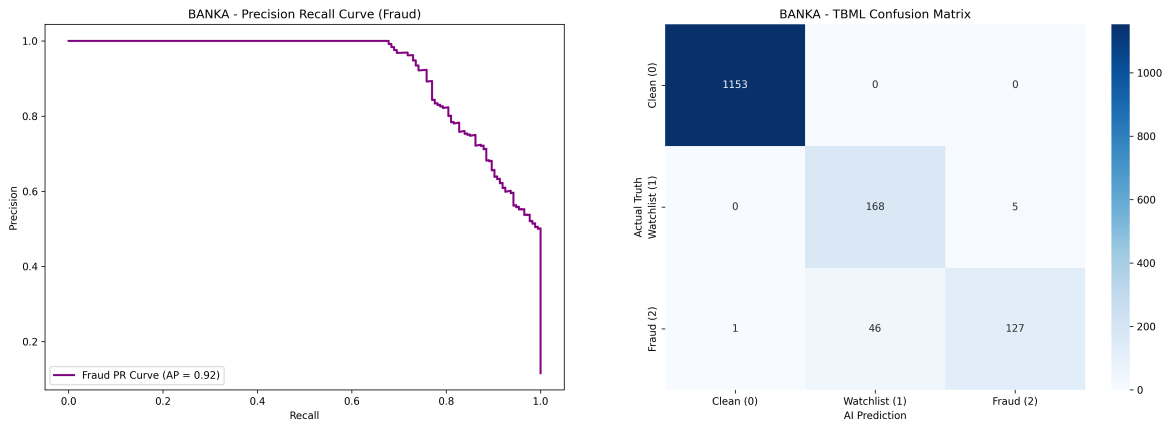

Figure 4.6: BANKA Precision–Recall Curve and Confusion Matrix Validation

As illustrated in Figure 4.6, the BANKA institutional environment achieved strong Precision–Recall analytical performance with an Average Precision (AP) score exceeding 0.90, thereby validating stable fraud-discrimination capability across minority transactional classes. The Precision–Recall curve demonstrated sustained precision stability across higher recall regions with limited degradation, indicating reliable identification of suspicious transactional activity under highly imbalanced fraud distributions. The confusion matrix additionally demonstrated near-perfect classification of legitimate transactional behavior with negligible clean-to-fraud misclassification propagation. Most classification overlap occurred between Watchlist and Fraud transaction states, where a lim-

ited number of high-risk transactions were classified interchangeably, thereby indicating successful learning of suspicious transactional topology patterns without excessive false-positive escalation during institutional fraud-monitoring workflows.

BANKB Precision–Recall and Confusion Matrix Validation

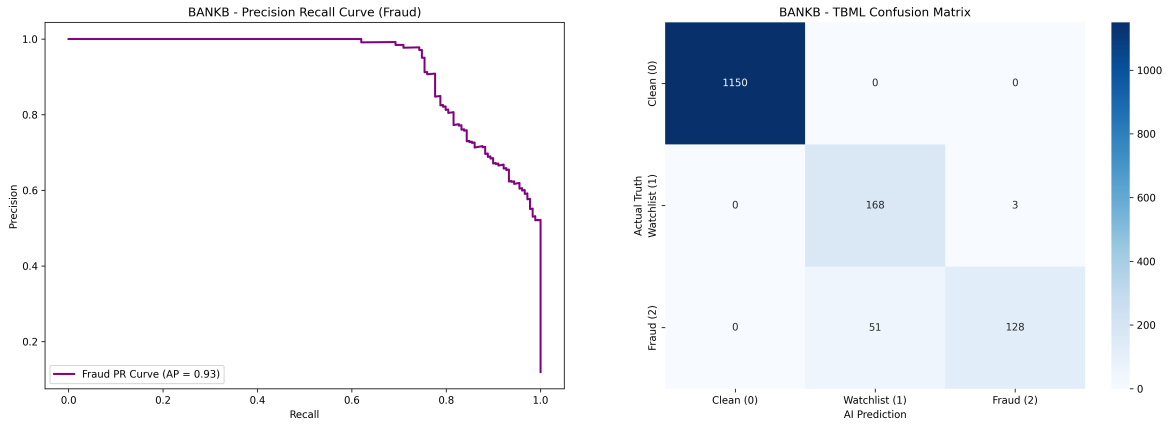

Figure 4.7: BANKB Precision–Recall Curve and Confusion Matrix Validation

As illustrated in Figure 4.7, the BANKB institutional environment demonstrated highly stable Precision–Recall behavior with an Average Precision (AP) score above 0.90 during federated analytical evaluation. The Precision–Recall curve maintained strong precision characteristics across progressive recall thresholds with controlled degradation at higher recall regions, thereby validating effective fraud-class sensitivity and reliable minority transaction detection capability. The confusion matrix further validated successful classification of legitimate transactional states with minimal clean-to-fraud prediction propagation. A comparatively small number of Fraud transactions were classified within the Watchlist category, indicating that the federated model preserved strong high-risk transactional discrimination while maintaining controlled false-positive behavior across institutional monitoring workflows.

BANKC Precision–Recall and Confusion Matrix Validation

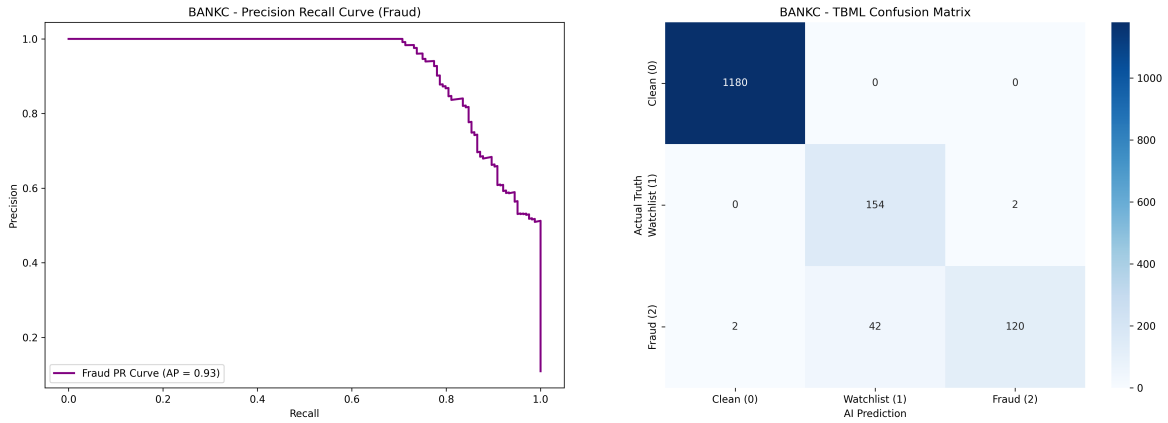

Figure 4.8: BANKC Precision–Recall Curve and Confusion Matrix Validation

As illustrated in Figure 4.8, the BANKC institutional environment achieved stable Precision–Recall analytical performance with an Average Precision (AP) score approaching 0.93, thereby validating reliable fraud-class discrimination across heterogeneous transactional environments. The Precision–Recall curve demonstrated sustained analytical stability with limited precision degradation across higher recall regions, indicating effective minority fraud-detection capability under imbalanced institutional transaction distributions. The confusion matrix additionally demonstrated strong classification consistency with near-perfect prediction of legitimate transactional behavior and minimal propagation of false-positive fraud classifications into clean transaction categories. Most prediction overlap remained localized between Watchlist and Fraud transaction states, thereby confirming stable federated learning behavior and effective distributed analytical generalization across isolated institutional transaction datasets.

4.3 Summary

The conducted testing and evaluation workflows validated the operational stability, synchronization reliability, and analytical performance of the proposed TBML detection framework across distributed institutional monitoring environments. The testing results confirmed stable backend communication, reliable transactional synchronization, controlled API response behavior, and successful workflow execution across graph-processing, federated analytical inference, dashboard synchronization, and compliance-management services. Concurrent scalability validation additionally demonstrated stable PostgreSQL query execution and controlled latency characteristics under increasing institutional work-

loads. The model performance evaluation further validated strong multi-class classification capability, stable federated convergence behavior, and reliable fraud-discrimination performance across heterogeneous institutional transaction environments without requiring centralized sharing of sensitive financial datasets.

Chapter 5

Results and Discussion

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the analytical outcomes and operational observations derived from experimental validation of the proposed federated Hybrid HGNN–LSTM framework for Trade-Based Money Laundering detection. Building upon the testing workflows discussed in Chapter 7, the following sections analyze the observed fraud-detection capability, federated learning stability, distributed synchronization behavior, and scalability characteristics of the deployed institutional monitoring framework.

5.1 Overview of Experimental Outcomes

The experimental evaluation demonstrated stable fraud-classification capability, reliable federated convergence behavior, and effective distributed synchronization across heterogeneous institutional transaction environments. The proposed framework successfully maintained multi-class analytical consistency for *Clean*, *Watchlist*, and *Fraud* transaction states while preserving institutional data isolation during federated optimization workflows.

The distributed streaming and graph-processing architecture additionally enabled synchronized real-time transaction propagation, stable heterogeneous graph generation, and controlled dashboard-update behavior during concurrent monitoring workflows. System-level scalability evaluation further demonstrated stable PostgreSQL query execution, efficient transaction persistence, and controlled API response latency under increasing institutional monitoring workloads.

5.2 Operational Performance Outcomes

The deployed monitoring framework demonstrated stable operational performance across distributed fraud-monitoring and compliance-management workflows. Dashboard transaction retrieval, fraud filtering, STR synchronization, and regulator-side compliance propagation maintained reliable execution consistency under concurrent institutional workloads. Table 5.1 summarizes the observed API-level operational response characteristics across major compliance workflows.

Table 5.1: Operational Workflow Performance Summary

Workflow	**Avg Response Time**	**Operational Observation**	**Status**
Dashboard Transaction Retrieval	$\sim$ 160 ms	Stable under concurrent workloads	Successful
Fraud Transaction Filtering	$\sim$ 148 ms	Controlled query latency during filtering	Successful
STR Compliance Retrieval	$\sim$ 466 ms	Stable relational aggregation behavior	Successful
STR Generation Workflow	$<$ 1 s	Successful compliance synchronization	Successful
Notification Propagation	$<$ 500 ms	Stable regulator-side synchronization	Successful
Federated Fraud Inference	$\sim$ 1–2 s	Stable graph-based analytical prediction	Successful

As observed in Table 5.1, dashboard synchronization and fraud-filtering workflows maintained comparatively low response latency during concurrent institutional workloads. The STR retrieval workflow exhibited comparatively higher latency due to computationally intensive relational joins, transaction-history aggregation, and compliance-report synchronization operations executed during report-generation workflows. However, stable response consistency and successful request execution were maintained across all monitored operational workflows.

5.3 Graph-Based TBML Detection Capability

The graph-based analytical workflow demonstrated strong capability in learning hidden transactional dependencies and multi-hop entity relationships across heterogeneous institutional transaction environments. Unlike traditional rule-based AML systems that primarily depend on isolated transactional heuristics, the proposed framework leveraged heterogeneous graph construction to capture interconnected relationships between ac-

counts, SWIFT transactions, trade metadata, compliance documents, and institutional entities during fraud-inference workflows.

The confusion-matrix evaluation discussed in Chapter 7 demonstrated that most classification overlap occurred between *Watchlist* and *Fraud* transaction states, while minimal propagation of fraudulent predictions into legitimate transactional categories was observed. This behavior validated the effectiveness of graph-topology learning for identifying suspicious transactional relationships without excessively escalating legitimate institutional transactions into fraud classifications during compliance-monitoring operations.

5.4 Federated Learning Effectiveness

The federated analytical workflow demonstrated stable distributed convergence behavior across all participating institutional clients during iterative FedAvg synchronization operations. The convergence progression analyzed in Chapter 7 showed gradual stabilization of Accuracy, Precision, and Recall metrics during later federated communication rounds, thereby validating effective distributed parameter synchronization and stable institutional knowledge sharing during collaborative optimization workflows.

Table 5.2 summarizes the observed federated analytical performance across distributed institutional clients after completion of iterative global aggregation workflows.

Table 5.2: Federated Analytical Performance Summary

Institution	**Transaction Class**	**Precision**	**Recall**	**F1-Score**
BANKA	Fraud (2)	0.96	0.73	0.83
BANKB	Fraud (2)	0.98	0.73	0.84
BANKC	Fraud (2)	0.98	0.73	0.84

As observed in Table 5.2, the federated Hybrid HGNN–LSTM framework maintained high Fraud-class Precision across all participating institutional environments while preserving stable Recall characteristics during distributed analytical inference. The observed convergence stability additionally indicated that the proposed framework successfully generalized fraud-learning behavior across heterogeneous institutional transaction environments without requiring centralization of sensitive transactional datasets.

5.5 Real-Time Monitoring and Compliance Outcomes

The distributed transaction-monitoring workflow demonstrated stable real-time synchronization of institutional transaction streams across heterogeneous backend analytical services. Kafka-based asynchronous event propagation enabled continuous ingestion of SWIFT transactional payloads into downstream graph-construction and federated inference pipelines without interrupting institutional monitoring workflows.

The automated STR generation and notification workflow additionally improved operational coordination between banking institutions and regulator-facing compliance environments. Fraud-classified transactional events were successfully transformed into structured compliance reports while preserving associated transaction histories, graph metadata, and institutional investigation references during distributed monitoring operations.

5.6 Scalability and Infrastructure Observations

Concurrent scalability evaluation demonstrated stable API response behavior and reliable PostgreSQL synchronization consistency under increasing institutional monitoring workloads. During dashboard transaction retrieval workflows, the framework maintained average response latencies near 156–162 ms across concurrent workloads ranging from 50 to 500 users while sustaining complete request success rates throughout the evaluation process. Similarly, fraud-transaction filtering workflows maintained average response latency near 148–159 ms under moderate concurrent workloads, thereby validating stable pooled-query execution and controlled backend synchronization during real-time monitoring operations.

The STR compliance retrieval workflow exhibited comparatively higher computational latency due to relational joins, transaction-history aggregation, and regulator-side compliance synchronization executed during report-generation workflows. Initial average response latency near 2694 ms during lower concurrent execution stages progressively reduced to nearly 466 ms during higher concurrent workloads, thereby indicating improved query-plan stabilization, PostgreSQL shared-buffer utilization, and optimized relational-query execution during repeated compliance-retrieval operations. Despite occasional tail-latency spikes during high-concurrency execution stages, the infrastructure evaluation maintained stable request consistency and complete request success rates throughout distributed monitoring and compliance-management workflows.

5.7 Summary

The experimental outcomes and analytical observations validated the effectiveness of the proposed federated Hybrid HGNN–LSTM framework for distributed Trade-Based Money Laundering detection and compliance monitoring. The observed results demonstrated stable graph-based fraud inference, reliable federated convergence behavior, controlled scalability characteristics, and effective real-time synchronization across distributed institutional monitoring environments. The framework additionally achieved privacy-preserving collaborative learning without requiring centralization of sensitive transactional datasets, thereby validating its operational suitability for large-scale institutional compliance ecosystems.

Chapter 6

Conclusion

CHAPTER 6

CONCLUSION

The proposed work successfully established a scalable and privacy-preserving federated graph-learning framework for intelligent Trade-Based Money Laundering detection across distributed institutional environments. The developed framework integrated heterogeneous graph construction, federated Hybrid HGNN–LSTM analytical inference, Apache Kafka-based transaction streaming, PostgreSQL-backed compliance orchestration, and regulator-facing monitoring synchronization within a unified operational architecture. Experimental evaluation demonstrated stable fraud-classification capability, reliable federated convergence behavior, and effective distributed synchronization across heterogeneous institutional transaction environments. The deployed framework achieved Fraud-class Precision values approaching 0.98 while maintaining stable Recall characteristics during federated analytical inference workflows. Concurrent scalability evaluation additionally demonstrated stable dashboard retrieval latency near 156–162 ms and controlled fraud-filtering response latency near 148–159 ms during increasing institutional monitoring workloads. The overall analytical and operational outcomes validated the suitability of the proposed framework for scalable, real-time, and privacy-aware TBML detection across distributed institutional compliance ecosystems.

6.1 Limitations

Although the proposed framework demonstrated stable analytical performance and reliable monitoring capability, certain practical and infrastructural limitations remain within the current implementation. The experimental evaluation primarily relied on synthetically enriched transactional datasets due to the limited availability of institutionally verified TBML datasets containing real-world laundering annotations. As a result, certain evolving laundering behaviors, cross-border transactional irregularities, and complex trade manipulation strategies may not be fully represented within the present experimental environment.

The federated analytical workflow additionally introduces communication overhead during distributed parameter synchronization and aggregation across participating institutional clients. As the number of banking environments increases, federated synchronization latency and distributed optimization complexity may proportionally increase

during large-scale deployment scenarios. Similarly, graph-processing workflows involving significantly larger transactional environments may introduce additional computational overhead during high-volume multi-hop graph traversal and distributed inference operations.

The current implementation is primarily focused on fraud detection, monitoring, and compliance-notification workflows. While the framework successfully supports regulator-side alert propagation and institutional investigation coordination, automated intervention mechanisms such as transaction blocking, account freezing, document-lock enforcement, and adaptive compliance-trigger execution are not directly integrated within the banking workflow. Such operational controls would require deeper integration with core banking infrastructure and institutional authorization pipelines.

6.2 Future Scope

Several opportunities remain for extending the proposed framework toward more scalable and production-ready financial crime-monitoring ecosystems. Future work can focus on validating the framework using larger institutionally verified TBML datasets containing real-world laundering annotations, cross-border trade behaviors, and dynamically evolving financial crime topologies. Such datasets can further improve analytical robustness and operational generalization across large-scale banking environments.

Future extensions may additionally explore transformer-based document intelligence frameworks capable of extracting semantic representations from unstructured trade documents including invoices, customs declarations, shipping records, and letters of credit. These contextual embeddings can subsequently be integrated into the graph-learning pipeline to strengthen semantic anomaly detection and trade-document verification workflows. The federated learning environment may also be extended toward geographically distributed banking ecosystems involving larger decentralized institutional participation.

Future implementations can additionally incorporate advanced secure aggregation mechanisms such as Fully Homomorphic Encryption (FHE), Secure Multi-Party Computation (SMPC), and differential privacy techniques to strengthen protection against gradient-leakage and model-inversion attacks during distributed parameter synchronization operations. Further research may also investigate Continuous-Time Dynamic Graph (CTDG) architectures and explainable graph neural network (XGNN) mechanisms to improve temporal topology tracking and graph-level interpretability for institutional com-

pliance analysts.

The proposed framework can further evolve into a fully integrated end-to-end banking compliance platform. Beyond fraud detection and notification propagation, future deployments may support automated compliance-response orchestration including transaction freezing, adaptive risk-trigger execution, institutional document verification workflows, regulator-approved intervention pipelines, and policy-driven account restriction mechanisms directly integrated with core banking infrastructure.

6.3 Learning Outcomes of the Project

The proposed TBML detection framework provided practical exposure to heterogeneous graph-learning architectures, federated learning environments, and real-time financial fraud-detection workflows. The project enabled hands-on experience with Graph Neural Networks, temporal sequence modeling, distributed parameter synchronization, and privacy-preserving analytical inference across institutional systems. In addition, expertise was developed in Apache Kafka-based streaming, PostgreSQL optimization, concurrent API scalability analysis, and full-stack integration using Next.js, Prisma ORM, and FastAPI. The implementation further strengthened understanding of AML compliance workflows, STR lifecycle management, fraud-monitoring operations, Precision–Recall evaluation, and distributed scalability validation within real-time analytical environments.

Appendix A

Plagiarism report

APPENDIX A

PLAGIARISM REPORT

Attach the screenshot of plagiarism report front page.

Appendix B

Publication details

APPENDIX B

PUBLICATION DETAILS

Attach the screenshot of either the Acknowledgment of the publication / Proof of Acceptance / Proof of Registration.

Appendix C

Appendix

APPENDIX C

APPENDIX

C.1 Hybrid HGNN–LSTM Model Implementation

The following listing presents the implementation of the proposed Hybrid HGNN–LSTM analytical model developed using PyTorch and PyTorch Geometric for federated Trade-Based Money Laundering detection.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import GATv2Conv, global_mean_pool

class TBML_DetectionModel(nn.Module):

    def __init__(
        self,
        kyc_dim=3,
        swift_dim=1,
        trade_dim=2,
        hidden_dim=32,
        lstm_hidden=32,
        classes=3
    ):

        super(TBML_DetectionModel, self).__init__()

        # Graph Branch

        self.gat1 = GATv2Conv(
            in_channels=kyc_dim,
            out_channels=hidden_dim,
            heads=2,
            concat=False
        )

        self.gat2 = GATv2Conv(
            in_channels=hidden_dim,
            out_channels=hidden_dim,
            heads=2,
            concat=False
        )
```

```
        self.gat3 = GATv2Conv(
            in_channels=hidden_dim,
            out_channels=hidden_dim,
            heads=2,
            concat=False
        )

        # Temporal Branch

        self.lstm = nn.LSTM(
            input_size=hidden_dim + swift_dim,
            hidden_size=lstm_hidden,
            batch_first=True
        )

        # Trade Branch

        self.trade_mlp = nn.Sequential(

            nn.Linear(trade_dim, 32),
            nn.LayerNorm(32),
            nn.ReLU(),
            nn.Dropout(0.2),

            nn.Linear(32, 32),
            nn.LayerNorm(32),
            nn.ReLU()

        )

        # Fusion Classifier

        self.classifier = nn.Sequential(

            nn.Linear(lstm_hidden + 32, 64),
            nn.LayerNorm(64),
            nn.ReLU(),
            nn.Dropout(0.2),

            nn.Linear(64, classes)

        )

    def forward(
        self,
        kyc_x,
        edge_index,
        swift_edge_attr,
```

```
        seq_data,
        trade_features
    ):

        # Graph Processing

        if edge_index.size(1) == 0:

            enriched_nodes = F.relu(
                self.gat1(kyc_x, edge_index)
            )

        else:

            x = F.dropout(
                F.relu(self.gat1(kyc_x, edge_index)),
                p=0.2,
                training=self.training
            )

            x = F.dropout(
                F.relu(self.gat2(x, edge_index)),
                p=0.2,
                training=self.training
            )

            enriched_nodes = F.dropout(
                F.relu(self.gat3(x, edge_index)),
                p=0.2,
                training=self.training
            )

        # Graph Pooling

        batch = torch.zeros(
            enriched_nodes.size(0),
            dtype=torch.long,
            device=enriched_nodes.device
        )

        graph_context = global_mean_pool(
            enriched_nodes,
            batch
        ).unsqueeze(1)

        # Sequence Shape Correction

        if seq_data.dim() == 4:
```

```
            seq_data = seq_data.squeeze(0)

        # Graph + Sequence Fusion

        graph_context_expanded = graph_context.expand(
            seq_data.size(0),
            seq_data.size(1),
            -1
        )

        combined_seq = torch.cat(
            [seq_data, graph_context_expanded],
            dim=-1
        )

        # Temporal Modeling

        lstm_out, _ = self.lstm(combined_seq)

        final_lstm_step = lstm_out[:, -1, :]

        final_lstm_step = F.dropout(
            final_lstm_step,
            p=0.3,
            training=self.training
        )

        # Trade Feature Processing

        trade_emb = self.trade_mlp(trade_features)

        # Final Fusion

        fused_vector = torch.cat(
            [final_lstm_step, trade_emb],
            dim=1
        )

        return self.classifier(fused_vector)
```

C.2 Federated Learning Server Implementation

The following listing presents the Flower-based federated server implementation used for distributed parameter aggregation and global model synchronization across institutional clients.

```
import numpy as numpy
import flwr as flwr

from model import TBML_DetectionModel


class modelStrategy(flwr.server.strategy.FedAvg):

    def aggregate_fit(
        self,
        curr_round,
        results,
        failures
    ):

        aggregated_weights, _ = super().aggregate_fit(
            curr_round,
            results,
            failures
        )

        if aggregated_weights is not None and curr_round == 10:

            print(
                f"Saving global weights for round {curr_round}"
            )

            # Convert Flower parameters to NumPy arrays

            aggregated_array = (
                flwr.common.parameters_to_ndarrays(
                    aggregated_weights
                )
            )

            # Save aggregated global weights

            numpy.savez(
                "global_model.npz",
                *aggregated_array
            )

            print(
                "Global weights saved successfully"
            )
```

```
        return aggregated_weights, _


if __name__ == "__main__":

    print(
        "Starting Flower server with custom strategy..."
    )

    # Initialize server-side model

    model = TBML_DetectionModel(
        kyc_dim=3,
        swift_dim=1,
        trade_dim=2,
        hidden_dim=32,
        lstm_hidden=32,
        classes=3
    )

    # Convert model weights to Flower parameters

    ndarrays = [

        val.cpu().numpy()

        for val in model.state_dict().values()

    ]

    initial_parameters = (
        flwr.common.ndarrays_to_parameters(
            ndarrays
        )
    )

    # Federated aggregation strategy

    strategy = modelStrategy(

        fraction_fit=1.0,
        min_fit_clients=3,
        min_available_clients=3,
        initial_parameters=initial_parameters

    )

    # Start Flower federated server
```

```
    flwr.server.start_server(

        server_address="0.0.0.0:8085",

        config=flwr.server.ServerConfig(
            num_rounds=10
        ),

        strategy=strategy

    )
```

C.3 Federated Learning Client Implementation

The following listing presents the Flower federated client implementation used for distributed institutional training, graph-based transaction processing, and federated evaluation within the proposed TBML detection framework.

```
import flwr as fl
import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import Dataset, DataLoader
from torch_geometric.nn import GATv2Conv, global_mean_pool

from neo4j import GraphDatabase
from collections import OrderedDict, Counter

import argparse
import random
import numpy as np

from model import TBML_DetectionModel

SEED = 42

torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

# Client Configuration
```

```
parser = argparse.ArgumentParser()

parser.add_argument(
    "--bank",
    type=str,
    required=True,
    choices=['banka', 'bankb', 'bankc']
)

args = parser.parse_args()

PORT_MAP = {

    "banka": 4000,
    "bankb": 4001,
    "bankc": 4002

}

MEMGRAPH_URI = (
    f"bolt://localhost:{PORT_MAP[args.bank]}"
)

MEMGRAPH_USER = ""
MEMGRAPH_PASS = ""

# Dataset Loader

class MemgraphTBMLDataset(Dataset):

    def __init__(
        self,
        uri,
        user,
        password,
        transaction_ids
    ):

        self.driver = GraphDatabase.driver(
            uri,
            auth=(user, password)
        )

        self.transaction_ids = transaction_ids

    def __len__(self):

        return len(self.transaction_ids)
```

```
def __getitem__(self, idx):

    target_msg_id = self.transaction_ids[idx]

    query = """
    MATCH (root:Transaction {id: $target_msg_id})
    OPTIONAL MATCH (root)-[:HAS_DOC]->(trade:Document)

    WITH root, trade

    MATCH (root)-[r1]-(n1:Account)

    WITH root, trade,
         collect(r1) as edges1,
         collect(n1) as nodes1

    UNWIND nodes1 as n1

    OPTIONAL MATCH (n1)-[r2]-(n2:Transaction)

    WITH root, trade,
         edges1,
         nodes1,
         collect(r2)[..50] as edges2,
         collect(n2)[..50] as nodes2

    WITH root, trade,
         nodes1 + [x IN nodes2
         WHERE x IS NOT NULL] AS all_nodes,

         edges1 + [x IN edges2
         WHERE x IS NOT NULL] AS all_edges

    UNWIND all_nodes AS n
    UNWIND all_edges AS r

    RETURN root,
           trade,
           collect(distinct n) AS nodes,
           collect(distinct r) AS edges
    """

    with self.driver.session() as session:

        result = session.run(
            query,
            target_msg_id=target_msg_id
```

```
            ).single()

        root = result["root"]
        trade = result["trade"]

        all_nodes_dict = {

            root.element_id: root

        }

        if trade:

            all_nodes_dict[
                trade.element_id
            ] = trade

        for n in result["nodes"]:

            all_nodes_dict[
                n.element_id
            ] = n

        nodes = list(all_nodes_dict.values())

        node_to_idx = {}
        kyc_features = []

        for i, node in enumerate(nodes):

            node_to_idx[node.element_id] = i

            if "Account" in node.labels:

                kyc = [

                    float(node.get("is_shell", 0)),
                    float(node.get("dorm_days", 0))/365.0,
                    float(node.get("device_entropy", 0))

                ]

            else:

                kyc = [0.0, 0.0, 0.0]

            kyc_features.append(kyc)
```

```
        kyc_x = torch.tensor(
            kyc_features,
            dtype=torch.float32
        )

        source_nodes = []
        target_nodes = []

        for edge in result["edges"]:

            start_id = edge.start_node.element_id
            end_id = edge.end_node.element_id

            if start_id in node_to_idx and end_id in node_to_idx:

                source_nodes.append(
                    node_to_idx[start_id]
                )

                target_nodes.append(
                    node_to_idx[end_id]
                )

        if len(source_nodes) == 0:

            edge_index = torch.empty(
                (2, 0),
                dtype=torch.long
            )

        else:

            edge_index = torch.tensor(
                [source_nodes, target_nodes],
                dtype=torch.long
            )

        raw_amount = np.log1p(
            float(root.get("amount", 0))
        )

        swift_edge_attr = torch.tensor(
            [[raw_amount]],
            dtype=torch.float32
        )

        seq_data = swift_edge_attr.unsqueeze(0)
```

```
        if trade:

            trade_features = torch.tensor([

                float(trade.get(
                    "price_deviation", 0.0
                )),

                float(trade.get(
                    "weight_gap_score", 0.0
                ))

            ], dtype=torch.float32)

        else:

            trade_features = torch.zeros(
                2,
                dtype=torch.float32
            )

        label = torch.tensor(
            int(root.get("label", 0)),
            dtype=torch.long
        )

        return (
            kyc_x,
            edge_index,
            swift_edge_attr,
            seq_data,
            trade_features,
            label
        )

# Federated Client

class BankClient(fl.client.NumPyClient):

    def __init__(
        self,
        model,
        train_loader,
        test_loader,
        class_weights
    ):

        self.model = model
```

```
        self.train_loader = train_loader
        self.test_loader = test_loader

        self.criterion = nn.CrossEntropyLoss(
            weight=class_weights
        )

        self.optimizer = optim.Adam(
            model.parameters(),
            lr=0.001
        )

    def get_parameters(self, config):

        return [

            val.cpu().numpy()

            for _, val in
            self.model.state_dict().items()

        ]

    def set_parameters(self, parameters):

        params_dict = zip(
            self.model.state_dict().keys(),
            parameters
        )

        state_dict = OrderedDict({

            k: torch.tensor(v)

            for k, v in params_dict

        })

        self.model.load_state_dict(
            state_dict,
            strict=True
        )

    def fit(self, parameters, config):

        self.set_parameters(parameters)
```

```
        self.model.train()

        for epoch in range(1):

            for (
                kyc_x,
                edge_index,
                swift_edge_attr,
                seq_data,
                trade_features,
                label

            ) in self.train_loader:

                kyc_x = kyc_x[0]
                edge_index = edge_index[0]

                swift_edge_attr = (
                    swift_edge_attr[0]
                )

                label = label[0]

                trade_features = (
                    trade_features[0].unsqueeze(0)
                )

                self.optimizer.zero_grad()

                logits = self.model(

                    kyc_x,
                    edge_index,
                    swift_edge_attr,
                    seq_data,
                    trade_features

                )

                loss = self.criterion(
                    logits,
                    label.unsqueeze(0)
                )

                loss.backward()

                torch.nn.utils.clip_grad_norm_(
                    self.model.parameters(),
```

```
                    max_norm=1.0
                )

                self.optimizer.step()

        return (
            self.get_parameters(config={}),
            len(self.train_loader.dataset),
            {}
        )

if __name__ == "__main__":

    train_dataset = MemgraphTBMLDataset(
        MEMGRAPH_URI,
        MEMGRAPH_USER,
        MEMGRAPH_PASS,
        []
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=1,
        shuffle=True
    )

    test_loader = DataLoader(
        train_dataset,
        batch_size=1,
        shuffle=False
    )

    model = TBML_DetectionModel()

    class_weights = torch.tensor(
        [1.0, 1.0, 1.0],
        dtype=torch.float32
    )

    client = BankClient(
        model,
        train_loader,
        test_loader,
        class_weights
    )

    fl.client.start_numpy_client(
        server_address="127.0.0.1:8085",
```

```
        client=client
    )
```

BIBLIOGRAPHY

[1] IMTF Compliance Technologies, *When trade becomes a cover: Unpacking trade-based money laundering*, 2024. [Online]. Available: https://imtf.com/blog/when-trade-becomes-a-cover-unpacking-trade-based-money-laundering/

[2] Financial Action Task Force and Egmont Group, "Trade-based money laundering: Trends and developments," FATF, Paris, France, Tech. Rep., 2020. [Online]. Available: https://fatf-gafi.org

[3] United Nations Office on Drugs and Crime, *Money laundering and Illicit financial flows overview*, 2024. [Online]. Available: https://unodc.org

[4] Europol, *Criminal finances, money laundering and asset recovery*, 2024. [Online]. Available: https://europa.eu

[5] J. Saenz and J. Lewer, "Quantifying illicit financial flows and trade misinvoicing in developing economies," *Journal of Financial Crime Research*, vol. 18, no. 2, pp. 112–128, 2024.

[6] J. Fan et al., "Deep learning approaches for anti-money laundering on mobile transactions: Review, framework, and directions," *IEEE Internet of Things Journal*, vol. 13, pp. 10 407–10 428, 2026, ISSN: 2327-4662. DOI: 10.1109/JIOT.2026.3653434

[7] A. I. Rasmus Ingemann Tuffveson Jensen, "Fighting money laundering with statistics and machine learning," *IEEE Access*, vol. 11, pp. 8889–8903, 2023, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2023.3239549

[8] J. Song, S. Zhang, J. Park, Y. Gu, and G. Yu, "Dimension expansion for learning money laundering activities hidden in transaction network," *IEEE Transactions on Computational Social Systems*, vol. 13, pp. 251–262, 2026, ISSN: 2329-924X. DOI: 10.1109/TCSS.2025.3599534

[9] J. Song, S. Zhang, P. Zhang, J. Park, Y. Gu, and G. Yu, "Illicit social accounts? anti-money laundering for transactional blockchains," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 391–404, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2024.3518068

[10] Z. Li, X. Yang, and C. Jiang, "Multi-view graph-based hierarchical representation learning for money laundering group detection," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2035–2050, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2025.3529321

[11] A. H. B. Gezici and E. Sefer, "Pagerank-based unsupervised deep vertex representations for anti-money laundering detection," *IEEE Access*, vol. 13, pp. 196 935–196 950, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3634197

[12] M. R. Karim, F. Hermsen, S. A. Chala, P. D. Perthuis, and A. Mandal, "Scalable semi-supervised graph learning techniques for anti money laundering," *IEEE Access*, vol. 12, pp. 50 012–50 029, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3383784

[13] K. Koo, M. Park, and B. Yoon, "A suspicious financial transaction detection model using autoencoder and risk-based approach," *IEEE Access*, vol. 12, pp. 68 926–68 939, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3399824

[14] A. E. Iyanda, "The role of misinvoicing in the money laundering cycle," *Journal of Anti-Corruption Law*, pp. 144–168, 2023. DOI: 10.14426/jacl.v3i1.1294

[15] A. E. Iyanda, "Principles and components of federated learning architectures," pp. 144–168, 2023. DOI: 10.14426/jacl.v3i1.1294

[16] K. Sheka, K. Victor, and E. Walker, "A synthetic data generation framework for money laundering detection using behavioral analysis learning approach in mobile banking systems," *ResearchGate*, Dec. 2024.

[17] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" In *International Conference on Learning Representations (ICLR)*, 2022. arXiv: 2105.14491.

[18] C. H. Kennedy, A. Hilal, and M. Momeni, "The role of federated learning in improving financial security: A survey," in *IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT)*, IEEE, 2025, pp. 1–8. arXiv: 2510.14991.